

Örebro universitet
Handelshögskolan - Informatik
Uppsatsarbete, 15 hp
Handledare: Andreas Ask
Examinator: Kai Wistrand
HT 2013

Testdriven utveckling

in action

Hur kan en organisation lyckas med
testdriven utveckling?

Pracha Karlsson (921129)
Magnus Lander (810216)
Daniel Mella (850223)

SAMMANFATTNING

Inom en stor del av all systemutveckling sker testerna av systemet som sista punkt innan systemet sjösätts. Testdriven utveckling är en systemutvecklingsmetod där testerna istället skrivs först och också är det som driver utvecklingen framåt.

Metoden höjs till skyarna av vissa och avfärdas omedelbart som onödigt omständigt av andra. Vi vill med denna uppsats undersöka hur det ser ut i verkligheten och vilka faktorer som påverkar användandet, inläringen och inställningen till testdriven utveckling.

Vi genomförde intervjuer på tre stycken Örebrobaserade organisationer och tittade utifrån ramverket *method-in-action* på vilka faktorer som påverkade användningen och varför..

Vi fann att utvecklarna närmade sig testdriven utveckling på väldigt olika sätt och grundade sin inställning mycket beroende på tidigare erfarenhet och inläring – oavsett hur lång eller kort den varit. Utvecklarna förväntas ofta bedriva självstudier utanför arbetstid – något som inte alltid funkar som kunskaputvecklingsform då tiden utanför jobbet ser olika ut beroende på var i livet man är. Det finns inte heller något klart program eller *best-practices* att följa för att lära sig metoden i någon av organisationerna. Vi såg också att det finns tekniker utanför själva metoden som utvecklare ganska omgående behöver bli bekanta med för att kunna utveckla testdrivet: *dependency injection* och *mock*.

Nyckelord: test-driven development, testdriven utveckling, red/green/refactor, *methods-in-action*, *dependency injection*, *mock*, systemutveckling, systemutvecklare

FÖRORD

Under tio intensiva, men intressanta och givande veckor under vintern 2013/14 har vi inom kursen för informatik C på Systemvetenskapliga programmet fördjupat oss i testdriven utveckling, method-in-action och alla aspekter runt omkring som påverkar användande och inläring.

Vi vill rikta ett speciellt tack till de tre Örebrobaserade organisationer som ställt upp med respondenter och till vår handledare Andreas Ask för vägledning.

Örebro, den 9 januari 2014

Pracha Karlsson

Magnus Lander

Daniel Mella

INNEHÅLLSFÖRTECKNING

1 INLEDNING.....	1
1.1 Bakgrund	1
1.2 Syfte	2
1.3 Problemformulering.....	2
1.4 Avgränsning.....	4
1.5 Intressenter.....	5
1.6 Perspektiv.....	5
1.6.1 Alternativa perspektiv	6
2 METOD	7
2.1 Datainsamling	7
2.1.1 Urval av intervjupersoner.....	8
2.1.2 Utformning av intervjufrågor	8
2.1.3 Etiska överväganden	9
2.1.4 Intervjuprocedur	9
2.1.5 Bortfall	9
2.2 Procedur för litteraturstudie	10
2.3 Analysmetod	11
2.4 Källkritik	11
2.5 Metodkritik	12
2.6 Validitet och reliabilitet	13
3 TEORI	14
3.1 Test-Driven Development	14
3.2 Stub och mock (fakes)	14
3.3 Dependency injection.....	15
3.4 Method-in-Action.....	15
3.4.1 Formaliserad metod.....	16
3.4.2 Metodens roll.....	16
3.4.3 Utvecklingskontext.....	17
3.4.4 Utvecklare	17
3.4.5 Information Processing System	18
3.4.6 Method-in-Action-molnet.....	19

4 RESULTAT OCH ANALYS.....	20
4.1 Respondenter	20
4.2 Utvecklare.....	20
4.2.1 Kunskap om TDD	20
4.2.2 Metodanvändning	21
4.2.3 Metodkunskap och metodmedvetenhet	22
4.2.4 Motivation och engagemang.....	22
4.3 Metodens roll	23
4.3.1 Standardisering av utvecklingsprocess	23
4.3.2 Systematisering av utvecklingskunskap.....	24
4.3.3 Möjliggöra projektstyrning & kontroll	25
4.3.4 Professionalisera SU-arbete	25
4.4 Utvecklingskontexten	25
4.4.1 Kontext och kultur	25
4.4.2 Filosofiska, praktiska och metodiska överväganden	26
4.5 Formaliserad metod	26
4.5.1 Verktyg, tekniker och designmönster	27
4.6 Informationssystem	27
5 AVSLUTNING	29
5.1 Slutsatser.....	29
5.1.1 Formaliserad metod kan utgöra grunden för en metod i användning	29
5.1.2 Metodens roll påverkar en metod i användning	29
5.1.3 Metodens roll rättfärdigar formaliserad metod.....	30
5.1.4 Utvecklingskontexten formar en metod i användning	30
5.1.5 Utvecklare analyserar utvecklingskontexten	30
5.1.6 Utvecklare utför en metod i användning.....	31
5.1.7 Utvecklare framställer informationssystem.....	31
5.2 Egna reflektioner	32
5.3 Vidareforskning	32
KÄLLFÖRTECKNING.....	33
BILAGOR	35
Bilaga 1, Intervjufrågor	35
Bilaga 2, Kodexempel: Hur stub och mock kan användas för att skriva automatiserade tester (i ramverket NUnit).	37
Bilaga 3, Kodexempel: Hur man applicerar dependency injection på kod	41

FIGURFÖRTECKNING

Figur 1. Arbetssättet red/green/refactor. Källa: egen, efter Beck (2003).

Figur 2. Ramverk för en metod i användning. Källa: egen, efter Fitzgerald et al. (2002).

Figur 3. Samband mellan en utvecklarens erfarenhet och nivå av metodanvändning. Källa: egen, efter Fitzgerald et al. (2002).

TABELLFÖRTECKNING

Tabell 1. Begreppslista.

Tabell 2. Sökord i litteraturstudien.

Tabell 3. Lista över respondenter.

BEGREPPSLISTA

BEGREPP	DEFINITION
Erfarenhet	Syftar på individens personliga kunskaper inom området programmering (beräknat på antal år) eller TDD (beroende på i vilket sammanhang vi använder ordet). Detta är någonting utvecklaren själv uppskattat.
Enhetstest	Innebär att man verifierar att en viss funktionalitet hos en enhet fungerar enligt förväntningarna. En enhet i det här fallet är det minsta "byggblocket" i en applikation som går att testa, oftast handlar det om en klass eller en metod.
Integrationstest	Innebär att man testar interaktioner mellan flera objekt. Med andra ord testar man <i>grupper</i> av objekt, vari enhetstestning innebär motsatsen till detta - där testas enskilda komponenter.
Metod, Systemutvecklingsmetod	Ett föreskrivet arbetssätt som beskriver hur man strukturerar, standardiserar och simplificerar systemutvecklingen och de ingående processerna. Detta genom att tala om vilka aktiviteter som ska genomföras och vilka tekniker som ska väljas.
SOLID	Beskriver fem grundläggande principer eller riktlinjer inom objektorienterad programmering. Kallas även för "first five principles" enligt Robert C. Martin.
GUI	<i>Graphical User Interface</i> , "grafiskt användargränssnitt". Ett gränssnitt som tillåter en användare att interagera med elektronisk utrustning.
Projekt	Tids- och omfattningsbegränsat uppdrag utfört av ett team för att framställa en unik produkt.
Förvaltning	Systemförvaltning. Att underhålla ett system.
Rött test	En indikation (inom ett test-ramverk) på att ett test misslyckats på grund av att det inmatade värdet inte överensstämde med en viss förväntning. " <i>Assert.AreEqual(1, 2)</i> " kommer till exempel visa ett rött test.
Automatiserat test	Test som utförs inom ett ramverk som specifikt är anpassat för detta.
IDE	"Integrated Development Environment", programvara avsedd för utveckling av olika typer av applikationer. Innehåller

	vanligtvis en textredigerare, kompilator och en debugger.
Systemutvecklingsprocess	Processbeskrivning för utveckling av system.
Utvecklare, Systemutvecklare	Personer som ansvarar för utveckling av IT-system. Omfattar en större folkgrupp: programmerare, projektledare, analytiker, systemarkitekter osv.
Behaviour-Driven Development	Systemutvecklingsprocess som är baserad på testdriven utveckling. BDD (Behaviour-Driven Development) kombinerar generella tekniker och principer från TDD. BDD beskriver genomförandet av ett program genom dess beteende utifrån sina intressenter.
Clean Code	En "tankesätt" som hjälper utvecklare att koda på ett lämpligare sätt som är mer förvaltningsbart.
Informationssystem	Ett system med IT-stöd som bearbetar information mellan diversa organisationer.

Tabell 1. Begreppslista.

1 INLEDNING

Vi kommer i detta kapitel att gå igenom bakgrunden till hur och varför testdriven utveckling, engelska test-driven development (TDD) uppkom. Sedan identifierar vi ett antal faktorer relaterat till TDD i en problemformulering som vi kommer att undersöka, och förklarar i samband med det vårt syfte med undersökningen. Därefter presenterar vi den avgränsning vi gjort, vem vi riktar resultatet av undersökningen till samt vilket perspektiv vi har haft och vilka andra perspektiv vi kan se.

1.1 Bakgrund

Sedan systemutvecklingens gryning har utvecklare följt en viss ordning av aktiviteter som tycks vara självklara när det gäller utveckling av informationssystem: *Design-Code-Test*. Ordet *design* betyder här att man planerar arbetet, som vad som ska göras och av vem. *Code* innebär att man exekverar planen: kod implementeras, applikationen byggs. Det hela avslutas med *test* bland annat för att säkerställa och verifiera att systemets funktionalitet uppfyller kravspecifikationen och att defekter identifieras. Detta kan ses som ett naturligt mönster att jobba efter, vilket har gjort att det används än idag (Lui & Chan, 2008). Testdriven utveckling innebär ett försök till paradigmskifte inom systemutvecklingsområdet. Det nya arbetssätt som TDD förespråkar har blivit känt som *Test-Code-Refactor* eller *Red/Green/Refactor*.

Kent Beck har lyfts fram som den person som ligger bakom idén med TDD även om han själv vidkänns att tanken inte är ny. Kollanus (2010) menar att testdriven utveckling har bedrivits sedan 1960-talet. Beck (2003) säger att TDD är en “medvetenhet om klyftan mellan beslut och återkoppling ... och tekniker för att styra klyftan”. Det innebär i det enklaste att skriva ett test, som kopplar ihop ett visst *beslut* (att skriva kod på ett visst sätt) med *återkoppling* (om koden gör det man tänkt).

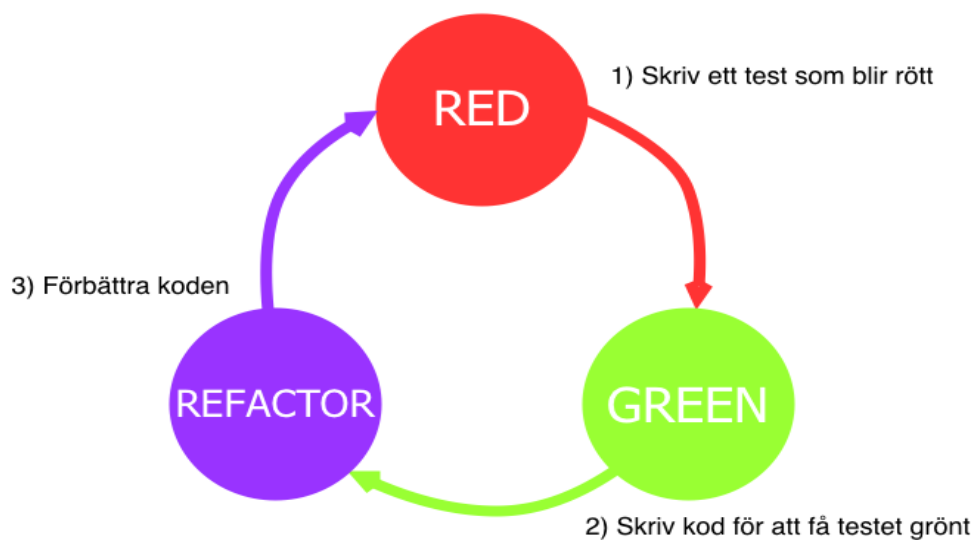
Enligt Beck (2003) används olika begrepp när man talar om TDD. Det ses som en metod (Tort et al., 2011), metodik (McDaid & Rust, 2009), en strategi (Janzen & Saiedian, 2005), en stil (Galloway, 2012), en vana (Hammond & Umphress, 2012), en design- och programmeringsdisciplin (Jeffries & Melnik, 2007) eller en systemutvecklingsprocess (Buffardi & Edwards, 2012).

Vi har valt att definiera TDD som en metod, enligt Goldkuhls (1991) definition, då dessa olika begrepp som används inte riktigt når fram med att beskriva vad TDD är. Det är inte bara en vana, en stil eller en strategi.

Metoder bygger på ett synsätt enligt Goldkuhl (1991, 1993) och består av *arbetssätt*, *begrepp* och *notation*. TDD som systemutvecklingsmetod är ett resultat av ett testdrivet synsätt. Riktlinjerna i metoden formar arbetssättet och handlar om vilka typer av frågor man ställer och beskriver därför hur man bör arbeta. Inom TDD utförs arbetssättet iterativt i en utvecklingsprocess efter mönstret *Red/Green/Refactor*. *Begrepp* finns både inom arbetssätt och notation och är de kategorier som ingår i typfrågorna och kan bland annat vara enhetstester och refaktorering.

Svaren på frågorna, det vill säga hur dessa olika delar uttrycks, görs med notation och innehåller enligt Goldkuhl (1991) semantik, uttrycksform och syntax. *Semantik* innebär de objekt som ska redogöras för (red, green, refactor). *Uttrycksformen* är inte uttryckligen fastställd men kan vara i löpande text eller som symboler i UML. *Syntax* är bestämda regler för hur modellen av Red/Green/Refactor ska se ut – till exempel att man inte får hoppa över något steg eller börja med att skriva kod för att få testet grönt.

Figur 1. Arbetssättet red/green/refactor.



Källa: egen, efter Beck (2003).

Vidare säger Beck (2003) att ordet *development* i begreppet TDD omfattar analys, logisk design, fysisk design, implementation, test, inspektion, integration och driftsättning – vilka är olika moment som ingår i en systemutvecklingsprocess. Med ordet *driven* menas att det är testerna som driver utvecklingen av systemet. *Test* syftar på automatiserade enhetstester, till exempel att testerna exekveras genom ett knapptryck.

1.2 Syfte

Syftet med denna studie är att närmare granska hur en organisation kan lyckas införa det testdrivna arbetssättet samt förstå hur de utvecklare som tillämpat TDD i de utvalda organisationerna resonerar kring metoden. Målet med uppsatsen är att ge organisationer underlag till att effektivt kunna införa TDD genom ett antal lösningsförslag.

Vår forskningsfråga blir därför: *Hur kan en organisation lyckas med testdriven utveckling?*

Vår målgrupp är systemutvecklare, projektledare och chefer som kommer att arbeta med TDD eller som har gjort det tidigare men inte varit nöjda med resultatet.

1.3 Problemformulering

För att kunna besvara vår forskningsfråga och uppfylla vårt syfte har vi ställt upp en specifik problemformulering:

Vilka problem och/eller svårigheter upplever utvecklare med TDD i användning?

För att i sin tur behandla vår problemformulering kommer vi att använda ramverket *method-in-action* (Fitzgerald, 2002) som hjälp för att ställa upp ett antal ytterligare delfrågor och för att kategorisera och relatera dessa frågor till varandra. Detta för att få en röd tråd genom hela studien. Modellen återkommer även under andra rubriker och förklaras ingående i teorikapitlet.

Utvecklare

Crispin (2006) menar att TDD kan upplevas som att ha en lång inlärningskurva bland en del utvecklare. Vidare menar Crispin (2006) att utbildningen (specifikt hur utvecklaren samlat sin kunskap) påverkar användningen av TDD. Fel i utbildningen kan leda till att metoden tillämpas på “fel” sätt. Då är det också svårt att förstå nyttan med TDD och varför man ska använda sig av det. Shore & Warden (2008) menar att det åtminstone krävs ett par månader med konstant användning av TDD för att man ska komma över “tröskeln”.

Vi finner därmed ett intresse av att undersöka *vilken utbildning utvecklaren haft samt utvecklarens personliga kunskap om TDD* och om dessa faktorer på något sätt kan påverka användningen av metoden.

Då vi själva provat på, visserligen i begränsad utsträckning, att tillämpa det testdrivna arbetssättet (med största fokus på att skriva enhetstester) och ibland upplevt att det varit svårt och mödosamt, vill vi även undersöka om *den personliga inställningen samt utvecklarens programmeringsfärdighet* kan påverka användningen av metoden.

Utvecklingskontext

Beck (2003) och Fitzgerald (2002) nämner att det finns en problematik i att leverans av programkod förväntas ske allt snabbare nuförtiden. En utvecklare som är stressad lägger generellt mindre tid på att göra tester. Det i sin tur leder till att utvecklaren gör fler fel, vilket leder till mer stress i en ond cirkel. Ebert (2007) visar på ett direkt samband mellan stressade utvecklare och ett ökat antal fel. Vi kommer därmed att undersöka om användningen av TDD kan påverkas av *stress eller leveranskrav*.

Formaliserad metod

En metod bör vara *formell* och *väldokumenterad*, därefter kan man kalla det för en formaliserad metod, menar Fitzgerald (2002). Vi vill därmed undersöka *hur utvecklaren arbetar med TDD* och jämföra det med det förespråkade arbetssättet som förklaras i teorin.

TDD bygger på enhetstester, därav namnet *Test-Driven Development*. Det innebär att metoden tillämpas i en utvecklingsmiljö (IDE, engelska: *Integrated Development Environment*) som har stöd för tester. Exempelvis Microsoft Visual Studio och det inbyggda testramverket MSTest. MSTest är ett av flera verktyg på marknaden som erbjuder stöd för automatiserade tester. Dessa typer av verktyg är någonting utvecklaren inte kan vara utan när hen tillämpar det testdrivna arbetssättet. Vi avser därmed att undersöka hur olika typer av *verktygsstöd* kan påverka användningen av TDD.

Metodens roll

Fitzgerald (2002) menar att en metod kan anta två olika typer av roller: *rationell* eller *politisk* (vilket förklaras ingående i teoriavsnittet), och att det finns vissa faktorer inom respektive roll som kan påverka användningen av den givna metoden. Fitzgerald (2002) nämner till exempel att en del utvecklare använder formaliserade metoder som ett sätt för dem att stärka sin ställning i teamet. Samtidigt är det ett sätt för de att verka "professionella" inom deras yrke (Martin, 2007). Vidare nämns att formaliserade metoder medverkar till någon sorts komfort eller självsäkerhet i teamet och att det kan ge auktoritet till vägval som ska göras.

Här är det därmed intressant att undersöka om *stöd från chef och ledning kan påverka användningen av TDD*.

Informationssystem

Bland de forskningsstudier som behandlar TDD som ämne har fokus oftast legat på nyutveckling och i mindre grad på vidareutveckling av existerande system (Shihab et al, 2011). Beroende på hur man definierar TDD och hur man ser på vidareutveckling kan det vara svårt eller till och med omöjligt att applicera det testdrivna arbetssättet på befintliga system då TDD i grund och botten handlar om att driva utvecklingen av ett systems design (eller arkitektur) framåt. Om koden då redan är skriven går det inte att utveckla dessa delar testdrivet utan att skriva om dem.

Vi finner här att det kan vara intressant att undersöka om *typ av applikation som skall utvecklas kan påverka användningen av TDD*.

1.4 Avgränsning

Vi har avgränsat studien till att undersöka utvecklarens upplevda problem och svårigheter med att tillämpa det testdrivna arbetssättet. Här fokuserar vi främst på de utvecklare som på något sätt arbetat med TDD, men det intresserar oss även att undersöka personer som kanske inte har tillämpat TDD i praktiken men som ändå har en viss kunskap om det (personer som läst teorin). Vi avgränsar oss även till att studera möjliga lösningsförslag till de problem som lyckats identifierats.

För att avgöra vilken relevans bakgrunden har så kommer vi att titta på utvecklarens tidigare erfarenheter med metoden samt hur utvecklaren blivit utbildad inom TDD. Och för att identifiera problem samt återge lösningsförslag så kommer vi att titta på hur utvecklaren omsätter metoden i praktiken.

Vi ville inte avgränsa oss till en organisation för att då enbart kunna säga något om den kontexten. Vi valde att fråga flera organisationer om dom var intresserade av att ställa upp med intervjupersoner. Genom att vill identifiera vilka problem som kan uppkomma med att använda TDD kan vi genom att intervjua personer från fler organisationer identifiera vad för typer av problem organisationer har med metodiken.

Då vi i två fall enbart intervjuat en person per organisation kände vi att vi inte kunde generalisera och jämföra hur organisationstyp eller -miljö inverkar eftersom denna person då skulle stått som ensam röst för organisationen.

I denna uppsats har vi inte gått in på djupet med *clean code* då detta är ett alldeles för brett område samt ligger utanför studiens huvudsakliga syfte. Vi uppmanar dock den intresserade läsaren att granska andra litteraturer som berör detta ämne då många principer inom TDD växt fram genom *clean code*.

Vi kommer inte heller att behandla alternativa metoder som är testbaserade, som till exempel *behavior-driven development* (BDD) då detta ämne befinner sig på en nivå högre än TDD. Vi problematiserar inte heller dependency injection, mock eller typer av designmönster eller om dessa verkligen är de bästa sätten att lösa avsedda problem på, bara för att vi ska kunna förhålla oss till TDD specifikt.

Vi anser att de ovanstående ämnena är värda att nämna på grund av att många litteraturer om TDD även behandlar något av dessa.

1.5 Intressenter

Våra huvudintressenter är organisationer och dess personal som upplevt problem med att införa TDD eller inte har infört det fullt ut på grund av en eller flera orsaker.

Genom att vi i denna uppsats kommer att försöka identifiera de bakomliggande orsakerna till problem och svårigheter med att tillämpa det testdrivna arbetssättet kan denna uppsats användas som stöd för utbildning inom TDD.

Resultatet kan också vara ett underlag till organisationer som har försökt införa TDD men som inte lyckats på grund av diverse orsaker. Organisationerna och utvecklarna kan ha idéer om varför det gick som det gjorde, utan att veta om det faktiskt stämmer eller om det kan bero på andra faktorer.

Andra intressenter kan vara andra studenter som själva är intresserade av TDD och som kan använda studien som empiri för att bättre underbygga sina egna teser och vidareutveckling för ny kunskap på området.

Vår egen kunskapsutveckling och intresse för TDD gör även oss själva till intressenter.

1.6 Perspektiv

Under en förintervju med Organisation A där vi diskuterade ämnesförslag för vår uppsats var en av de tänkbara problemställningarna varför Organisation A hade försökt, men misslyckats med att införa TDD som ett arbetssätt i sina projekt. I vår litteraturstudie försökte vi hitta faktorer som kan ha påverkat användandet av TDD men fann ingen sådan studie. Därmed uppmärksammades vi på varför inte litteraturen inte har tagit upp liknade problem och problemområdet blev därtill ett intresseområde för vår forskning.

Vi ser TDD som en svårförståelig metodik (Crispin, 2006), men tror inte att detta är hela sanningen till varför TDD inte används. Vi tittar därför på andra aspekter, framförallt sociala, som kan påverka användandet. Vilka aspekter har berott delvis på våra egna föreställningar och delvis vilka aspekter som blivit tydliga under intervjuerna.

Method-in-Action ramverket är det perspektiv som vi kommer att utgå ifrån. För att identifiera vilka aspekter som påverkar TDD i användning kommer vi att formulera intervjufrågor utifrån ramverkets komponenter och därmed få en helhetsbild om vad som kan påverka metoden.

Ingen av oss har tidigare tillämpat TDD i något riktigt systemutvecklingsprojekt. Däremot har vi provat på att göra enhetstester. Den testdrivna utvecklingen upplever vi som omständig och vi ser att det krävs mycket kunskap för att kunna skriva ett bra test.

Den kunskap vi tagit fram genom litteraturgenomgången tyder på att det finns märkbara fördelar med att använda TDD. Vi har dock haft svårt att faktiskt se fördelarna som litteraturen talar om och upplever att inlärningskurvan är brant och att testerna drar ner på produktionstakten.

Vi har i en tidigare kurs tittat på ett framförallt tre teorier kring anpassning av systemutvecklingsmetoder: *situationell metodkonstruktion*, *metodkonfiguration* och *method-in-action*.

1.6.1 Alternativa perspektiv

Vi kan se ett antal alternativa perspektiv, varav kanske det mest uppenbara är just det *traditionella* sättet att utveckla. *Design-Code-Test* där testandet kommer som sista punkt i systemutvecklingen. Då skulle man kunna titta på vad det innebär för ett utvecklingsprojekts framgång och förvaltningsbarhet att testerna sker sist. Genomförs testmomentet alltid eller har det medvetet lagts sist för att kunna tas bort vid tidsbrist? Vad innebär det i så fall för antalet fel som finns i systemet och i slutändan vad innebär det för systemets stabilitet och vidareutvecklingsmöjligheter? Sker det mycket nyutveckling på grund av att system som utvecklas utan att testas ordentligt inte är tillräckligt förvaltningsbara? Att vi inte tar upp detta perspektiv mer än översiktligt beror på att vårt syfte är att undersöka TDD, inte att göra en jämförande studie.

Det är också möjligt att se ett alternativt perspektiv från en högre chefs eller ledningspersons perspektiv, där man kanske framförallt tittar krasst resultatenriktat på ekonomin. Är det verkligen värt de extra arbetstimmar att arbeta testdrivet eller ens testa mer än det absolut mest grundläggande?

Ur ett arbetsmiljöperspektiv, vilket vi delvis tar upp, där man främst fokuserar på de ökade kraven, ökad stress på grund av detta och om det är värt att införa TDD om man ställer vinsterna med TDD mot nackdelarna för personerna som ska arbeta med det. Vad innebär det för en utvecklarens hälsa att införa fler arbetsmoment i en redan arbetstyngd vardag?

2 METOD

I detta kapitel kommer vi att redogöra för det tillvägagångssätt vi har använt oss av i vår undersökning. Hur vi har samlat in data beskrivs ingående med vilka vi har valt att intervjua och varför, hur vi har utarbetat intervjufrågorna, hur intervjuerna har gått till, etiska reflektioner och bortfallet. Vi går sedan in på hur vi genomförde vår inledande kunskapsutveckling via en litteraturstudie och hur vi analyserade det insamlade materialet. Sist i kapitlet tar vi upp hur vi förhållit oss till källkritik, metodkritik samt validitet och reliabilitet.

2.1 Datainsamling

Enligt Oates (2006) kan datainsamling bestå av två olika typer av data: primärdata och sekundärdata. Det förstnämnda är det som är knutet till den faktiska undersökningen och är den kunskap som tagits fram av författarna själva, genom exempelvis intervjuer och enkäter. Det sistnämnda består av befintlig kunskap, det vill säga kunskap som någon annan tagit fram. Exempel på sekundärkällor kan vara artiklar, böcker och rapporter av andra författare eller forskare.

Vi har valt att göra en kvalitativ undersökning med muntliga intervjuer som vår primära datakälla. En kvalitativ studie valdes då syftet med vår uppsats är att skapa en förståelse kring problemområdet. Men även eftersom att vi söker detaljerad information, vi vill få svar på komplexa frågor och fånga personliga uttryck som annars är svåra att fånga upp i en kvantitativ undersökning (Oates, 2006).

En kvalitativ intervju kan enligt Bryman (2011) bestå av två olika huvudtyper: strukturerad och kvalitativ. Den kvalitativa typen innefattar ostrukturerade och semi-strukturerade intervjuer. *Strukturerad* innebär att man har fasta frågor (frågor som inte är tänkt att kunna förändras) som kan användas mot samtliga respondenter. *Kvalitativ* innebär snarare att man saknar fasta frågor och att det är en öppen diskussion där respondenten kan svara hur hen vill. Vår intervju är av en semi-strukturerad typ, vilket innebär att frågor kan ställas i oberoende ordning. Andra eller omformulerade frågor kan även ställas mot respondenten beroende på situationen (Oates, 2006). Vi valde denna typ av intervjuer då vi inte i förväg kände till vilka aspekter som låg till grund för problematiken vi skulle undersöka.

Vi utförde intervjuerna på respondenternas olika arbetsplatser som Oates (2006) menar underlättar för dem och får dem att känna sig mer bekväma och därför blir mer öppna med sina svar. Vi var i fem av de sex intervjuerna tre personer som intervjuade en respondent, och i en intervju var vi två personer som intervjuade en respondent.

2.1.1 Urval av intervjupersoner

Vi tog mejlkontakt med sju Örebrobaserade organisationer som vi vet arbetar med systemutveckling. Då vi hade som kriterium för vår undersökning att respondenterna skulle ha arbetat med TDD tidigare gjorde vi ett målinriktat urval (Bryman, 2011). Organisationernas kontaktpersoner ansvarade själva för processen att få tag på personer i organisationen med den efterfrågade profilen.

Vårt urval bestod av sex stycken systemutvecklare från tre olika organisationer: Organisation A som ställde upp med fyra intervjupersoner, Organisation B med en person och Organisation C med en person. Urvalet blev en blandning av personer som arbetat länge i yrket och nyanställda, personer som hade någorlunda stor erfarenhet av TDD (minst ett år) och personer som enbart hade berört det i mindre projekt som de hade jobbat med på fritiden.

2.1.2 Utformning av intervjufrågor

Vi utformade en kvalitativ undersökning med öppna frågor då det enligt Oates (2006) tillåter respondenterna att prata mer fritt och passar vår ansats som är *orsaksinventerande*. Det vill säga, vi känner till effekten (att det finns tvekan till att arbeta med TDD) men vill undersöka orsakerna till det (Goldkuhl, 2011).

Alla frågor ställdes inte på alla intervjuer och vissa frågor som ställdes finns inte med i underlaget till intervjufrågorna då de ställdes som följdfrågor för att fånga upp något visst som sades under intervjun (Oates, 2006). Vissa av intervjufrågorna ställdes, omformulerade och av mer uppföljande karaktär, en andra gång. Oates (2006) menar att detta kan användas för att försöka locka fram ett mer uttömmande svar utifrån den kontext som diskuteras.

Intervjufrågorna utformades utifrån det perspektiv och den problematik med TDD som uppdagades under vårt första möte med Organisation A. Detta kopplade vi sedan till delen av vår litteraturstudie som behandlade *method-in-action*, varpå vi fick fram ett antal problemformuleringar som vi utformade intervjufrågor utifrån. Frågorna har delats in i tre olika kategorier: 1) *Utgångsläge och perspektiv*, 2) *Inläring och erfarenhet* och 3) *Användning*.

I *utgångsläge och perspektiv* tar vi reda på antal år i yrket, inställning till TDD med mera för att få en bild av deras bakgrund och perspektiv. Detta tillåter oss att lägga grunden för att studera *method-in-action*-komponenten *Utvecklarna*.

Därefter går vi in på *inläring och erfarenhet* där vi ställer frågor om hur deras inlärningsprocess sett ut, hur stor erfarenhet de har och börjar komma in på deras inställning till TDD. Detta för att vidare se hur de närmar sig TDD och om olika typer av inläring påverkar användningen av TDD. Vi får då reda på mer om komponenten *Utvecklarna*.

Under *användning* har vi ställt merparten av frågorna. Detta då det är problematik i användning (komponenten *method-in-action*-molnet) och hur komponenterna i *method-in-action* är relaterade till varandra och hur kopplingarna mellan dem formar användningen som är vårt syfte. Här har vi ställt frågor som berör den kontext som utvecklarna är i (komponenten *Utvecklingskontext*), vilken syn (komponenten *Roll*) utvecklarna tillskriver TDD, hur de hanterar TDD som formaliserad metod (komponenten *Formaliserad metod*) och

vad som utvecklas (komponenten *Informationssystem*). Dessa frågor berör de faktorer vi identifierat (se 1.2 *Problemformulering*) som kan vara bidragande till framgångsrikt införa TDD i en organisation.

Intervjufrågorna återfinns som Bilaga 1.

2.1.3 Etiska överväganden

Vetenskapsrådet (2002), som är en myndighet under Utbildningsdepartementet, och som har som uppgift att “utveckla svensk forskning av högsta vetenskapliga kvalitet”, har ställt upp fyra huvudkrav på forskningen när den berör individer på något sätt. Dessa är informationskravet, samtyckeskravet, konfidentialitetskravet och nyttjandekravet. Kraven är identiska med de som ställts upp av Bryman (2011) och Oates (2006).

Vi uppfyllde *informationskravet* genom att informera respondenterna vid starten av varje intervju vad vår undersökning gick ut på, deras roll i den samt frågade om vi fick spela in intervjun för att mer korrekt kunna återge vad som sagts. Vi informerade även dem att deras svar kommer att avrapporteras anonymt.

Samtyckeskravet menar vi har uppfyllts då vi personligen inte har ställt några krav på respondenternas deltagande i intervjuerna. Det kan dock inte uteslutas att det kommit krav på deltagande ovanifrån i de respektive organisationerna.

För att uppfylla *konfidentialitetskravet* har vi sett till att de inspelade ljudfilerna samt transkriberingarna enbart är delade med oss själva. Vi har också oidentifierat organisationerna i rapporten.

För att uppfylla *nyttjandekravet* kommer vi inte att använda de insamlade uppgifterna till annat än forskningsändamål. Detta informerades också om vid starten av varje intervju.

2.1.4 Intervjuprocedur

Intervjuerna utfördes med vår egen medverkan och enbart en respondent i taget. Tid och plats valdes ut i förväg. Samtliga intervjuer gjordes på respondenternas arbetsplatser då det blev mer bekvämt för respondenterna (Oates, 2006). Proceduren genomfördes initialt med en presentation av studien där vi berättade om vårt ämne och upplägget för själva intervjun. Varje intervju pågick i ungefär 60 minuter. Eftersom intervjuerna var semistrukturerade (Oates, 2006) uppkom ytterligare frågor eller följdfrågor.

2.1.5 Bortfall

Det totala bortfallet var fyra organisationer och två respondenter. Tre av organisationerna som tillfrågades att medverka i undersökningen tackade nej eftersom ingen hade arbetat med TDD. En annan organisation hörde aldrig av sig tillbaka. En av respondenternas deltagande avböjdes då beskedet om hens vilja att delta kom sent och vid en tidpunkt då vi redan hade uppnått en mättnad i studien. Den andra respondenten genomgick en intervju med oss, men då hen saknade erfarenhet och kunskap om TDD kunde vi inte komma fram till något som kunde anses som relevant för studien.

2.2 Procedur för litteraturstudie

Bryman (2011) påpekar betydelsen av en litteraturstudie för att inte göra onödigt arbete och uppfinna hjulet på nytt. Vi gjorde en systematisk litteraturstudie enligt riktlinjer uppsatta av Bryman (2011) och började med att 1) kartlägga vårt syfte och 2) definiera de kriterier vi ansåg behövdes ställas upp för att hålla litteraturstudien inom de frågeställningar vi tidigare ställt upp. Vi gjorde sedan en litteraturgenomgång inom de ramar vi satt upp i punkt 1 och 2.

Till vår studie har böcker och artiklar använts som sekundära källor.

Litteraturen fick vi fram genom sökningar med Summon, Scopus, LIBRIS och Google. De två förstnämnda databaserna valdes ut på grund av dess databas av vetenskapligt granskade artiklar. Detta är artiklar som andra forskare har granskat och anses därför ha högre trovärdighet än annat material. Summon är en aggregator, som sammanställer artiklar, böcker med mera från många olika källor – bland annat Scopus. LIBRIS innehåller enbart böcker. Betydligt färre artiklar finns indexerade hos Summon och Scopus än hos Google. I de fall vi behövde mer information valde vi därför att söka via Google. Vi var väl medvetna om att vi skulle få många träffar på tidningsartiklar, forumtrådar och bloggar – som nästan alla inte är vetenskapligt granskade. Dessa har därför inte använts som källor, om de inte också fanns bland de vetenskapligt granskade artiklarna.

Då vårt huvudområde är testdriven utveckling började vi med sökorden “test driven development” i Summon. Vårt syfte i detta läge var att skaffa en helhetsbild över ämnet. Nedan beskriver vi hur sökningarna gjordes i de olika databaserna.

Databas	Sökord	Antal träffar	Utvalda träffar
Summon	test driven development	44	21
LIBRIS	test driven development	78	4
Google	test driven development	1 620 000	0
Scopus	test driven development AND software engineering	178	0
LIBRIS	methods in action	9	1

Tabell 2. Sökord i litteraturstudien.

I databaserna Summon och Scopus kunde vi filtrera sökresultaten. Vi var enbart intresserade av vetenskapligt granskade publikationer som kunde visas med fulltext.

I Summon har vi begränsat ytterligare genom att ange ämnesordet “Test-Driven Development” för att då få med publikationer som behandlar TDD specifikt. Detta resulterade i 44 träffar. Vi diskuterade i gruppen och nådde konsensus att av dessa fanns 21 för ämnesområdet och vårt syfte relevanta träffar, vilka lästes i fulltext. Genom den avgränsning vi gjort kunde vi bestämma vad som var relevant. Vi sammanfattade dessa artiklar samt antecknade titel, länk, år, resultat av studien och nyckelord för varje artikel.

I databasen Scopus har vi använt sökorden "*test driven development*" AND "*software engineering*" i sökfälten rubrik, abstract och nyckelord – vilket resulterade i 178 träffar. För att hålla oss till vårt undersökningsområde har vi begränsat oss till resultat inom ämnesområdena *Computer Science* ("datavetenskap") och *Social Science* ("samhällsvetenskap") och dokument av typerna *Article* och *Conference Paper*, vilket gav 119 träffar. Dessa har lästs översiktligt (titel, abstract). Vi fann inga fler relevanta artiklar, för vårt syfte och avgränsning, i Scopus än de vi redan hade hittat i Summon.

En strategi vi tog till var att leta artiklar i referenslistor hos andras uppsatser vilket har resulterat i att vi hittat böcker som *Test Driven Development by Example* av Kent Beck. Vi fann även annan litteratur av samma författare, *eXtreme Programming Explained* som tar upp TDD i användning tillsammans med systemutvecklingsmetoden eXtreme Programming.

I LIBRIS fann vi inga fler böcker än de vi redan hade hittat genom att titta på de vetenskapligt granskade artiklarnas referenslistor.

2.3 Analysmetod

När vi hade genomfört samtliga intervjuer transkriberade vi våra inspelningar.

Enligt Oates (2006) ska man ha ett gemensamt format när man jämför. Därför skrev vi ut transkriberingarna på papper och klippte sedan upp dessa i delar beroende på vad de besvarade – för att det skulle vara lättare för oss att sammanställa svaren och identifiera problem. Dessa färgkodades samtidigt beroende på vilken respondent som svaret tillhörde. Detta för att kunna bibehålla konfidentialitetskravet.

För att kategorisera svaren tog vi hjälp av Fitzgeralds (2002) ramverk *Method-in-action*. Ramverket tar upp aspekter som påverkar när en metod är *i användning* (Fitzgerald, 2002). Metoder kan variera mellan att iaktta de tekniska aspekterna till att iaktta de mer humanistiska aspekterna där de sociala faktorerna har större fokus (Fitzgerald, 2002).

Vi kategoriserade analysen kring ramverkets komponenter och grupperade svaren utifrån: Vad kan utvecklarna om den *formaliserade metoden*, vad för *roll* kommer metoden ha för utvecklarna, vilka erfarenheter har *utvecklaren*, i vilken *kontext* kommer metoden att användas i och vilka förbättringar kommer metoden att medföra ett *informationssystem*. Ramverket hjälper oss att få en bättre förståelse för hur metoden TDD kan komma att användas. Analysarbetet kommer då att jämföras med litteratur och identifiera vilken problematik TDD medför i och med att en utvecklare använder metoden. Därtill kan vi urskilja vilka relationer komponenterna har till varandra och hur metodens "verkliga" användning ser ut.

2.4 Källkritik

Thurén (2003) har ställt upp fyra källkritiska principer: äkthet, tidsaspekt, beroende och tendens. *Äkthet* innebär att den/det som källan utger sig för att vara också är sant i realiteten. *Tidsaspekten* säger att desto nyare en källa är desto mer trovärdig är den, vilket speciellt gäller när utsagan omfattar många detaljer. Det finns samtidigt ett flertal komplicerande aspekter

som måste tas hänsyn till, såsom personens ålder, intresse samt fragmenterade eller förvrängda minnesbilder. *Beroende* går ut på att man ska använda sig av förstahandskällor (*vertikalt perspektiv*) och att det krävs minst två av varandra oberoende källor för att ett påstående ska kunna anses vara trovärdigt (*horisontellt perspektiv*). Slutligen *tendens*, som handlar om att vara medveten om att de som kan ha ett intresse av att ljuga också kanske gör det.

I vår litteraturstudie har vi enbart valt artiklar från vetenskapligt granskade publikationer. Vi förlitar oss på att denna referentgranskning har gjorts enligt rådande process och är tillförlitlig och har därför inte gjort någon egen granskning av dessa artiklar.

I enstaka fall har vi använt artiklar som inte är vetenskapligt granskade. I samtliga fall är dessa dock skrivna av författare med flera publikationer och andra granskade artiklar bakom sig. *Tendensprincipen* är något vi speciellt har fått förhålla oss till när det gäller dessa artiklar.

De böcker vi har använt är inte vetenskapligt granskade men är utgivna av välkända bokförlag inom IT-området och skrivna av författare med omfattande erfarenhet på området. Flera av böckerna används som kurslitteratur på Örebro universitet.

Under intervjuerna har vi försökt att inte ställa vägledande frågor som kan skeva svaren. Vi har försökt att skapa en miljö där respondenterna känt sig bekväma och haft möjlighet att vara uppriktiga (se 2.1.3 *Etiska överväganden*). Dessa källor (respondenterna) är förstahandskällor och uppfyller principerna om äkthet och tidsaspekt. Tendensprincipen har behandlats genom att vissa intervjufrågor har ställts omformulerade en andra gång – i detta fall för att ge oss en bättre möjlighet att ta reda på om respondenten verkligen är insatt i det omfrågade eller av olika anledningar försökt dölja sin okunskap.

2.5 Metodkritik

Vi inledde vår uppsats med en litteraturstudie för att få en bredare kunskap om TDD och närliggande koncept. Vi anser att detta var det bästa sättet att skapa en bild av kunskapsområdet och om vad som har och vad som inte redan har studerats.

Intervjuerna genomfördes med öppna frågor för att bättre fånga upp problematiken. Vi utgår i intervjufrågorna och analysen från *method-in-action*-modellen, med fokus på vad som hände när TDD skulle tillämpas. Samtidigt tittar vi också särskilt på utvecklaren och dess inläring, det vill säga de aspekter som förekom TDD *i användning*. Vi skulle ha kunna gjort en omfattande enkätundersökning och samtidigt riktat oss till fler personer (nationellt och internationellt). Men då vi för det första upplevt att det är få organisationer som använt TDD och för det andra att problematiken vi ville undersöka berodde på aspekter som ännu inte kartlagts, kände vi att vikten av att kunna ställa följdfrågor och i större grad få mer djupgående svar skulle ge oss ett bättre underlag än en skriftlig datainsamling (Oates, 2006).

Vilken metod som är bäst att använda är svårt att uttala sig om då de skiljer sig i många aspekter och passar olika bra in på olika områden.

2.6 Validitet och reliabilitet

Oates (2006) säger att validitet avser hur pass relevant något är i ett visst sammanhang och att det handlar om att välja rätt saker i rätt situationer. Exempelvis kan det vara relevant att ta upp enhetstester när man diskuterar TDD. För att uppnå en hög validitet och visa på genomskinlighet har vi bifogat de intervjufrågor vi använt samt beskrivit vår metod så ingående som varit möjligt. Vi har också haft en nedskriven och systematisk approach i vårt arbete och beskrivit hur vi har tänkt och värdesatt i olika situationer och överväganden som uppkommit under arbetets gång. Hur vi har utformat vår intervjuundersökning, vad vi har tagit ställning till och de val vi gjort finns återgivna i 2.1 Datainsamling och underrubrikerna.

Reliabilitet innebär hur pass pålitligt något är, att mätningarna är korrekt utförda. Ett experiment som gjorts på ett urval personer ska gå att repetera på samma grupp individer och alltid generera samma resultat (Oates, 2006). Detta kan dock vara svårt att uppnå eftersom människor kan ändra sina åsikter med tiden. För vår undersökning är det svårt med reliabilitet då individerna vi intervjuat kan ha lärt sig mer om TDD eller fått annan kunskap som påverkar deras framtida svar. Det kan även vara så att det vedertagna arbetssättet för systemutveckling ändrats avsevärt när undersökningen ska repeteras och därför får helt andra slutsatser.

3 TEORI

Vi kommer i detta kapitel att beskriva teorin bakom de delar vi valt ut att studera från litteraturstudien och som vi har använt oss av som grund för vår uppsats.

3.1 Test-Driven Development

Test-driven development (TDD) definieras enligt Beck (2003) som en utvecklingsstil, “a style of development” medan Jeffries (2007) väljer att kalla det för en programmeringsdisciplin. Andra författare menar att TDD egentligen borde kallas för “test-driven design” och inte för “test-driven development” (Buschmann, 2011). Martin (2007), även känd som “Uncle Bob”, definierar det såhär “TDD är en disciplin som hjälper utvecklaren att leverera ren och flexibel kod som fungerar”.

TDD lägger stor fokus på enhetstester och använder dessa för att driva utvecklingen av systemets design framåt. Konceptet med TDD är enkelt: Skriv ett test som inte kommer att fungera. Gör sedan den minsta åtgärden som krävs för att testet ska fungera. Refaktorera sedan. Dessa arbetsregler samlas under begreppet *Red/Green/Refactor* (Beck, 2003) och som förklaras mer detaljerat nedan.

Red: Utvecklaren börjar med att fundera på vilka krav en viss operation ska ha. Detta hjälper hen att verkligen tänka igenom hur koden ska konstrueras (kom ihåg att TDD ska driva systemdesignen). Därefter skriver utvecklaren ett test på några få rader och förväntar sig att det ska bli rött, eftersom ingen kod har implementerats ännu. Det är även fullt giltigt att ett kompilationsfel uppstår, enligt Beck (2003).

Green: Utvecklaren implementerar därefter tillräckligt med kod för att få testet att bli grönt. Återigen ska implementationen endast innefatta några få rader kod. Det är okej att lösningen hårdkodas eftersom refaktorering sker härnäst.

Refactor: Refaktorering handlar om att strukturera om innehållet i en metod utan att förändra dess yttre beteende (Fowler, 1999). Refaktorering definieras enligt Oshero (2013) som en handling att förändra ett kodavsnitts design utan att ha sönder befintlig funktionalitet. Här finns möjlighet för utvecklaren att finjustera eller “städa” koden genom att eliminera duplikationer, förändra namn på variabler, rensa onödig kod med mera.

Denna cykel repeteras sedan och för varje test som ska skrivas.

3.2 Stub och mock (fakes)

En stub ersätter ett beroende hos en klass. Stubs gör det möjligt att testa koden utan att behöva interagera med dess *verkliga* beroenden (Oshero (2013)). Mock, eller mer korrekt “mockobjekt”, definieras som “fejkade” objekt och dessa används för att verifiera ett beteende hos en klass (Oshero (2013)) (Fowler, 2007). Man bör skilja på begreppen *mock* och *stub* då dessa, vilka kan tyckas likna varandra, i själva verket är två olika saker. Det finns även någonting som heter *fake*, detta är en term som används för att beskriva dels mock eller stub, beroende på i vilket sammanhang det sker i (Oshero (2013)).

I Bilaga 3 har vi sammanställt ett kodexempel för att illustrera hur man kan använda mocks och stubs under enhetstestning.

3.3 Dependency injection

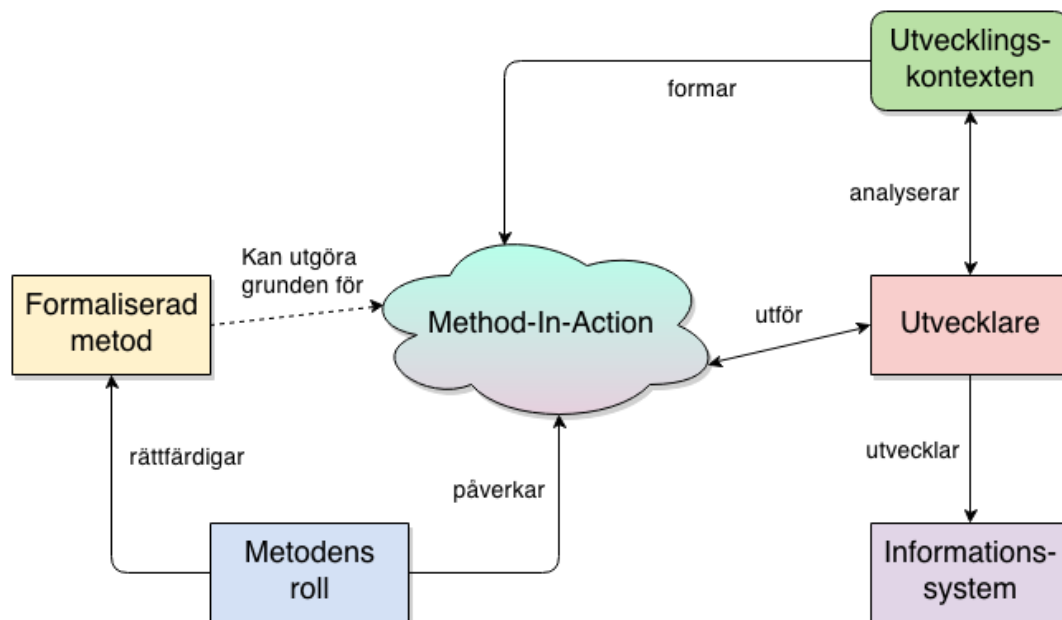
Dependency Injection (DI) är ett designmönster och är ett bland flera sätt som gör det möjligt att skapa lösa kopplingar mellan olika klasser. Det beskrivs även i en av SOLID's principer "Dependency Inversion Principle". DIP menar att en klass ska vara fri från konkreta beroenden och istället kommunicera med abstrakta implementationer, eller interfaces. DI som applicerats på en klass innebär att klassen inte längre behöver ansvara för att instantiera dess egna beroenden. Detta är även någonting som förespråkas av SOLID's "Single Responsibility Principle" som menar att en klass ska ha ett enda ansvarsområde (Martin, 2008).

Vi har i bilaga 3 konstruerat ett kodexempel för att illustrera hur DI kan användas.

3.4 Method-in-Action

Method-in-Action, *metod i användning*, är ett ramverk som illustrerar den komplexa karaktären som en metod antar och de komponenter som påverkar metoden i användning. Ramverket illustrerar vilken typ av systemutveckling som pågår och tar upp viktiga aspekter och faktorer som måste beaktas. Samtidigt som ramverket visar systemets helhet är det aldrig möjligt att fullständigt definiera någon av komponenterna utan att ta med hela ramen Fitzgerald (2002).

Figur 2. Ramverk för en metod i användning.



Källa: egen, efter Fitzgerald et al. (2002).

Fitzgerald (2002) menar att method-in-action inte är en metod utan ett ramverk som gör det möjligt för utvecklare och forskare att se över den komplexa processen som influerar ramverkets olika komponenter och deras integration med varandra.

3.4.1 Formaliserad metod

Fitzgerald (2002) säger att formella metoder är en samling av olika av metoder som uppfyller de kriterier som kommer att vara ett fokus inom systemutveckling. Grundtanken med formaliserade metoder är främst att ge stöd för utvecklingsprocessen – för en mer effektiv, säkrare, mer förutsägbar och lättare process att kontrollera.

Formaliserad metod eller bara *metod* är ett samlingsnamn på olika typer av systemutvecklingsmetoder. Exempelvis RUP (*Rational Unified Proccess*), XP (*eXtreme Programming*), Scrum, men även TDD (Fitzgerald, 2002).

3.4.2 Metodens roll

En viktig del av ramverket är metodens roll inom utveckling. Här finns det två breda men diametralt motsatta roller som metoden kommer att ha inom utvecklingsprocessen.

Fitzgerald (2002) nämner första kategorin som är *rationella förklaringar* och som tar upp det intellektuella och argumentation varför en metod ska användas. Dessa rationella förklaringar kan delas upp i olika roller som belyser vilka faktorer som är viktiga. En sådan roll kan bestå av att kunna bryta ut en utvecklingsprocess och dela in den i olika kategorier i mindre och i mer passande steg för att reducera komplexiteten i en systemutvecklingsprocess och på så sätt få större kontroll. På så sätt har projektledningen kontroll över utvecklingsprocessen.

Fitzgerald (2002) fortsätter med att en annan roll är *uppdelning av arbetskraft* och här tas upp aspekter som framhäver ekonomiska overseenden av arbetskraft där utvecklingsprocessen kräver olika kompetensnivå och därtill olika lönenivåer. Arbetsgivaren får då tillgång till en bättre ekonomisk överblick och kan fördela resurserna där de bäst behövs. Målet med *systematisering av utvecklingskunskap* syftar till att ändra alla beroende av kunskaper till objektsform, där kunskap kan lagras, systematiseras, spridas och utbytas. Kunskap kan då överföras från erfarna till oerfarna utvecklare och minska på inlärningskurvan av utvecklingsprocesser. Genom att *standardisera utvecklingsprocess* förbättrar man samordningen och kommunikationen mellan utvecklarna i komplexa projekt och avlastar underhållningsarbetet.

Den andra kategorin, säger Fitzgerald (2002), handlar om de *politiska förklaringar* som en metod kan ha. Dessa roller påverkar metodens bakomliggande faktorer varför metoden används och vilka influenser metoden kan medföra under utvecklingsprocessen.

Fitzgerald (2002) menar att metoder kan bidra till mer “professionell” utveckling och medföra bättre organisationsplanering och strategiformulering för IT-avdelningar. Ledningen får en ökad komfort, förutsägbarhet och förtroende för utvecklingen om arbetssättet följs. Genom att metoder för med sig någon typ av dokumentation under utvecklingsprocessen finns det möjlighet att beslut i framtida processer kan fattas på säkrare grunder. Det finns tendenser att utvecklare försöker göra anspråk på metoder för att vinna kontrakt med statliga organisationer, göra reklam för viktiga intressenter eller att försöka uppnå ISO-certifiering. Det finns möjligheter att metodens roll kan påverka att en utvecklarens profil höjs inom organisationen, utan att det är för organisationens bästa.

3.4.3 Utvecklingskontext

Komponenten utvecklingskontext framhäver komplexiteten och den dynamiska karaktären av en organisation där utvecklingen utförs. Kontexten berör även de tekniker och arbetssätt (föreskrivna regler) som tillämpas i organisationen. Enligt Fitzgerald (2002) är alla informationssystem belägna i och utvecklade i en kontext. Med detta menas att informationssystem är skapade, designade och producerade för specifika ändamål i en specifik miljö. I många fall talar man att kontext är en organisation.

Kontexten, säger Fitzgerald (2002), tar upp aspekter som se förhållanden mellan förändringar i kontexten och vad för förändringar som kan påverka kontexten med teknik. Organisationer ställs dagligen mot att nya tekniker uppkommer och måste välja hur de ska anpassa sig till det. Ny teknik gör att den interna kompetensen förändras, hur man gör uppgifter förändras, nya produkter utvecklas och nya sätt att utnyttja befintlig kompetens.

I och med detta kan kontexten förändras genom organisationens kultur. Inställningen om vilken roll som kulturen spelar och hur utvecklarna beter sig beror på tro och värderingar.

Fitzgerald (2002) fortsätter med att inom varje kontext finns det tradition och kultur som framhäver vilken typ av angreppsstrategi organisationen vill använda för att närma förändring och det finns olika strategier genom vilka informationssystem används för att åstadkomma förändring. Dessa förändringsstrategier kan vara proaktiva, som letar efter möjligheter, kontra reaktiva, som löser uppkomna problem. Problemlösningar där systemutveckling som fokuserar på analysen kontra innovation där systemutvecklingen är en kreativ process. Inkrementella förändringar som är mindre riskabelt och hanteras i små steg kontra radikala förändringar. Långsiktig formaliserade systemutvecklingsprojekt kontra kortsiktiga systemutvecklingsprojekt. Strategi med hög risk där radikala förändringar med oprövad teknik kontrasteras mot strategier med låg risk och beprövade lösningar.

Som det har tagits upp tidigare så är kontexten unik för varje organisation. Beroende på strategi som har valts måste man veta hur kontexten ser ut, förstå den och vad som påverkar.

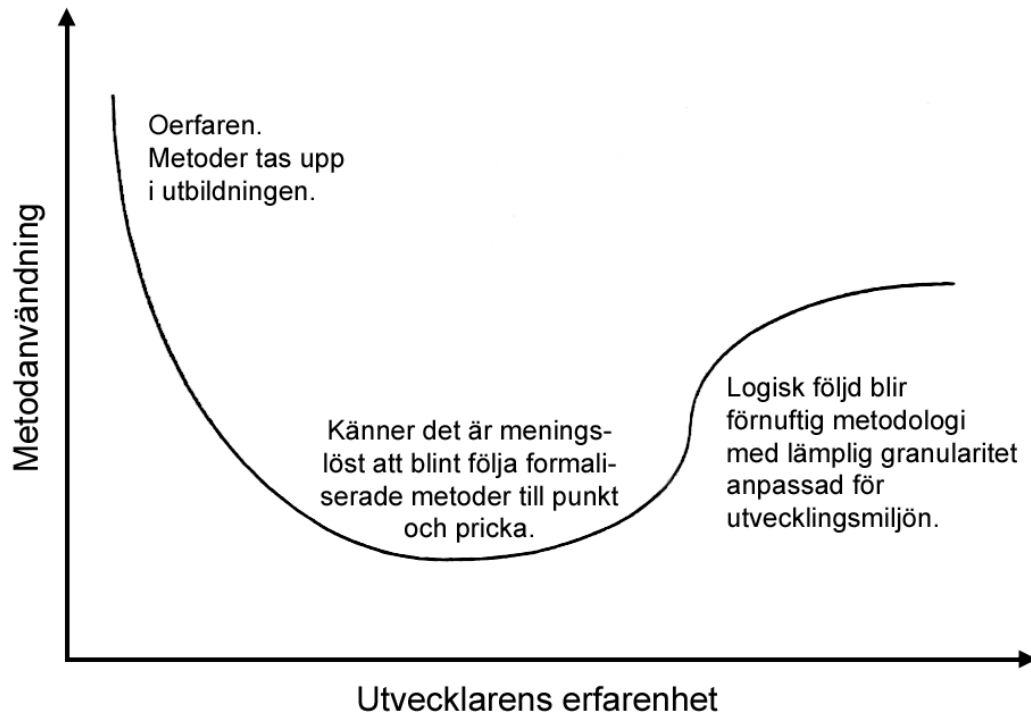
3.4.4 Utvecklare

En annan komponent i ramverket är utvecklare. Termen används i bred bemärkelse, och täcker en mångfald av aktörer, systemanvändare, analytiker, designerns, programmerare, kunder och intressenter. Denna komponent återspeglar att det är människor och inte metoden som utvecklar systemet. Metoder är enbart ett ramverk som hjälper individer att få fram ett resultat – men det är kunskapen om metoden som får resultatet att bli bra (Fitzgerald, 2002).

En utvecklares kunskap ligger till grund för lyckad systemutveckling. Fitzgerald (2002) menar att metoder har tendenser att begränsa en utvecklares förmåga och att utvecklaren inte kommer att utveckla sin kreativa sida på samma sätt. Här tas också upp att hur en utvecklares utveckling erfarenhetsmässigt kommer att förändra uppfattningen av behovet av metodanvändning. I detta sammanhang påpekar Fitzgerald (2002) att metoder inte framhäver den grundläggande kunskapen om applikationer och att man kan förlita sig för mycket på den tekniska expertisen. En faktor som tas upp är utvecklarens förmåga att vara engagerad och motiverad för arbetet de utför.

Komponenten tar upp åsikter om en utvecklarens erfarenhet påverkar metodanvändning. I figur 3 beskrivs förhållandet mellan oerfarna utvecklare och de med mer erfarenhet i relation till metodanvändning. Figuren säger att en oerfaren utvecklare kommer att använda metoder på grund av komplexiteten i utvecklingen medan en erfaren utvecklare inser de fördelar som en metod ger, samtidigt som det är större risk att utvecklaren känner sig begränsad av metodanvändningen.

Figur 3. Samband mellan en utvecklarens erfarenhet och nivå av metodanvändning.



Källa: egen, efter Fitzgerald et al. (2002).

3.4.5 Information Processing System

Ett informationssystem riktar sig till konstruktion, produktion och genomförande av ett system i en kontext men samtidigt har ett socialt och ett tekniskt synsätt. Den mer traditionella utvecklingen speglar den ingenjörslignade strukturen för att utveckla och lösa problem men senare insåg man att den sociala biten var minst lika viktig. Till vilken grad som utgör ett informationssystem kan aldrig vara helt definierad. Genomförandet av ett system kan stödja valet av en metod, bestämma vad för typ av kompetens och skicklighet är nödvändigt och hur det ska behandlas med kontexten (Fitzgerald, 2002).

Fitzgerald (2002) identifierade olika typer av så kallade systemfamiljer. En systemfamilj är en grupp system som har liknade drag, egenskaper och syfte. Dessa systemfamiljer är därför en viktig del av Method-in-Action för varje systemfamilj kommer då att påverka hur utvecklingsprocessen kommer att formas.

3.4.6 Method-in-Action-molnet

I praktiken är det sällan att metoden är tillämpad i sin helhet och inte heller som metodskaparen har tänkt sig även om den kan ge en mall för vägledning i praktiken. Man kan utgå från en formaliserad metod *som kan utgöra grunden för* metodens användning, men det är inte helt nödvändigt enligt Fitzgerald (2002). Författaren menar vidare att utvecklare inte *utför* metoder på samma sätt på grund av att olika utvecklare tolkar och tillämpar metoder präglade av den erfarenhet de har. Utvecklarna kommer inte heller att tillämpa samma metod på samma sätt beroende hur utvecklingssammanhanget har *format* hur metoden ska användas och vad för roll metoden har som kan *påverka* metodens användningsområde. Alla utvecklingsprojekt är unika, sett från Method-in-Action.

4 RESULTAT OCH ANALYS

I denna del börjar vi med att presentera våra respondenter översiktligt. Därefter lägger vi fram en sammanfattning av intervju svaren vi fått och analyserar dem utifrån vår problemformulering och vårt syfte – baserat på vilken del i vårt valda analysramverk vi har kategoriserat dem i.

4.1 Respondenter

Respondent	Organisation	Jobbtitel	År i yrket	Antal projekt med TDD	SU-metod för TDD	Antal projekt med automatiska tester
1	B	Lead developer	8	2 stora	Scrum + XP	2
2	A	Lead developer	4	2	Scrum (anpassad)	3
3	A	Systemutvecklare	15	0	-	1
4	C	Lead developer	4	5	Scrum + XP	10
5	A	Konsult	25	1	Scrum	3
6	A	Junior developer	1	1	Scrum	2

Tabell 3. Lista över respondenter.

Respondent 3 hade inte arbetat med TDD i något “riktigt” projekt men vi valde ändå att inkludera respondenten eftersom hen sedan tidigare hade provat TDD, fast då i ett projekt som gjordes på fritiden.

4.2 Utvecklare

Metoder används och tolkas på olika sätt (Fitzgerald, 2002). Här tar vi upp hur utvecklarna utnyttjar metodens principer och hur metoden kommer att påverka utvecklarnas förmåga. Avsnittet tar upp utvecklarens kunskapsområde och erfarenhet om metoden.

4.2.1 Kunskap om TDD

Självstudier är den främsta inlärningstypen bland utvecklare, och fem av sex respondenter nämner detta som huvudsaklig form av inlärning. Samtidigt är det vanligt att gå en kurs i

ämnet antingen på universitet, på arbetsplatsen eller genom arbetsgivaren. Flera nämner att de tyckte det var bra att få veta att det dem lärt sig via självstudier också kunde bekräftas av en kunnig person, såsom en lärare eller författare på området.

Vissa har arbetat med TDD i ett par år, men två respondenter tar upp att de enbart har begränsad erfarenhet av TDD i praktiken; “några dagar” till “ett par veckor” säger båda. Respondent 5 påpekar också att “kunskapen om TDD varierar väldigt mycket bland folk”.

Beck (2003) nämner att TDD är ett luddigt begrepp och genom våra intervjuer har vi sett att det till en början var långtifrån självklart vad TDD verkligen innebar. Med bakgrund av det är det tydligt att det finns ett behov av teoristudier i början av inläringen – men inte hur mycket som helst. Flera utvecklare nämner att de gärna vill fort in i praktiska exempel eftersom det är så de lär sig bäst. Detta kan innebära att den som skall läras upp får sitta med en enkel applikation som utvecklas testdrivet. Vikten av att det inte är en alltför komplex applikation bör poängteras, för att utvecklaren ska kunna fokusera på TDD i sig och inte behöva lära sig något annat nytt samtidigt. Några som arbetat lite längre med TDD rekommenderar att man inte bör angripa själva TDD direkt, utan snarare börja med att lära sig automatiserade tester.

Vikten av att ha en expert att rådfråga är en central del för att lyckas med TDD och samtliga respondenter tar upp detta. Respondent 2 påpekar att det måste vara en person på plats och som “... man kan få svar från snabbt och inte tre veckor senare.” Dock är det enbart en respondent som uttryckligen påpekar att hen har haft tillgång till en sådan expert. Några respondenter tar upp parprogrammering som ett möjligt alternativ. Parprogrammering innebär att två personer arbetar vid samma dator; en förare som skriver koden (som då kan mer om TDD) och en observatör som reflekterar över det som skrivs och de bestämmer sig tillsammans hur de ska ta sig framåt.

4.2.2 Metodanvändning

Det finns en stor variation i hur mycket utvecklarna har arbetat med TDD. Flera av respondenterna säger att de implementerar automatiserade enhetstester och/eller integrationstester snarare än arbeta med testdrivet. Ingen av de vi intervjuat arbetar med TDD dagligen eller i alla projekt de jobbar med. “Det är sällsynt att se någon i projektet arbeta testdrivet”, påpekar en respondent. Några nämner att de försöker arbeta med TDD så ofta de kan medan resterande säger att de arbetar med TDD i vissa projekt. I en respondents team var det bara några få personer som arbetade med TDD och många av de andra utvecklarna hade aldrig provat det överhuvudtaget. TDD är inget som prioriteras i någon av de organisationer vi undersökt och om det ingår alls så kommer det längre ner i kravspecifikationen.

Det är vanligt bland oerfarna utvecklare att man skriver en mångfald av “onödiga” tester eftersom man inte vet vad som är viktigt att testa, enligt ett några respondenter. Till detta menade en respondent att “... testerna skrivs, men de ger inte utslag”. När en utvecklare har greppat konceptet med TDD kommer nästa utmaning: att lära sig att skriva bra test. Det ingår inte i TDD, men är en förutsättning för att det ska vara meningsfullt.

Alla respondenter tar på ett eller annat sätt upp hur de själva upplever att andras inställning till TDD ser ut. Hos samtliga handlar det om hur det finns personer som ställer sig tvekan till att använda TDD. Exempelvis respondent 4 säger att “det finns folk som har gamla vanor och

inte är motiverade att lära sig TDD”. Respondent 2 säger att “nya [utvecklare] har lättare att tänka i nya banor” och att “det är svårare att ändra på personer som jobbat längre då de har arbetat på ett visst sätt under lång tid” och ställer sig frågande till varför det nya sättet skulle vara bättre. Respondenten säger också att det ofta är de senast nyexaminerade som är mest entusiastiska för ny teknik och metoder. Detta kan sättas i relation till figur 2 som visar på sambandet mellan en utvecklarens erfarenhet och nivå av metodanvändning.

4.2.3 Metodkunskap och metodmedvetenhet

De som arbetat minst med TDD påpekar att det är svårt, så pass svårt att det till och med fått dem att sluta med det. Svårigheterna ligger bland annat i att man inte riktigt förstår själva metoden som förklaras i teorin. Som till exempel varför man ska skriva ett rött test (se 3.2 Red/Green/Refactor). “Man vet ju precis hur man ska göra ändå, det är frustrerande att arbeta så”, menade en. Samma person medger att det kan ha med dennes oerfarenhet inom TDD att göra.

Vissa utvecklare tycker att TDD är onödigt omständigt och ser inte vad det ger för mervärde. En del tycker att integrationstester räcker och menar att enhetstester tar längre tid att implementera. Därför upplevs det som “overkill”. Mot detta inflikar en respondent kort och gott att “många säger att TDD är dåligt, men dom vet inte vad dom pratar om”. De som väljer bort TDD för att enbart implementera enhetstester/integrationstester har olika skäl till det, men i flera fall handlar det om mognadsgrad – att de bara har arbetat med TDD under kort tid. I de fall där en utvecklare som har arbetat längre med TDD men ändå väljer bort det för implementation av enhetstester/integrationstester så görs det av en av två anledningar: kostnad eller typ av projekt.

Det är sällan tillräckligt att förstå TDD och kunna arbeta testdrivet. Man kommer ganska omgående även att behöva bekanta sig med andra saker som anses vara nödvändiga. Det kan då upplevas som en lång och spretig inlärningsprocess än man ursprungligen tänkt. Det handlar till exempel om att bli bekant med tekniker som *mock* och *dependency injection*. Respondent 2 nämner att “ingen vet vad mocka betyder” och att det kan vara “svårt att greppa för att man ska kunna utföra tester, som att man behöver dependency injection för att det ska funka”.

4.2.4 Motivation och engagemang

“Det handlar mycket om eget intresse. TDD låter mer komplicerat än det behöver vara”, säger en respondent.

Engagemanget för att arbeta testdrivet varierar kraftigt bland respondenterna. En tredjedel är mestadels positiva och resten är antingen neutrala eller delvis negativt inställda. Respondent 2, som har mer erfarenhet och en annan roll i organisationen, är positiv bland annat då “man tjänar på det längre fram” eftersom utvecklingskostnaden för ett projekt enbart är “10 % och förvaltningen 80-90 %”. Hen tillägger att det blir “lättare att göra ändringar i systemet, och det kommer även kunna leva längre för man kommer kunna refaktorera mycket lättare på grund av att man har sina tester”. Respondent 5 tror att TDD “tar längre tid än kunden har lust att betala för” – något som ytterligare två respondenter instämmer i. Respondenten förklarar

att hen inte “har avfärdat [TDD] på något sätt, men inte omfamnat det heller”. Respondenten tyckte att det var omständigt att arbeta testdrivet och menar att en av anledningarna till hens inställning beror på att det tar längre tid.

När man talar om eget intresse så har det framkommit att vissa utvecklare upplever att de saknar motivation att lära sig TDD. Det beror dels på den långa startsträckan som många talar om, men också svårigheter med att förändra sitt vanliga arbetssätt; att bryta gamla vanor. Detta är desto vanligare bland äldre utvecklare som har fler år i yrket än bland de yngre och mer nyligen nyexaminerade. Insikten att personer med olika bakgrund lär sig på olika sätt och behöver olika mycket tid och resurser är därför viktig att komma fram till.

För att få med sig alla utvecklare, särskilt de som kanske redan har en negativ bild av TDD, nämner samtliga vi intervjuat att det är viktigt att visa på fördelarna med TDD jämfört med hur de arbetar idag. Detta snarare än att introducera det som ett nytt koncept som “ska” användas och tycks komma uppifrån utan att vara förankrat i hela organisationen. Respondent 1 säger till exempel att “det är viktigt att alla förstår fördelarna, att man inte gör tester bara för att någon annan säger det”. Speciellt då för seniora utvecklare då dessa är vana att arbeta på ett visst sätt och som respondent 6 uttrycker det “har mycket annat [att göra], så dom har inte tid till att utvecklas [på samma sätt som en nyexaminerad]”. Respondent 4 fyller på vidare att den leveransansvarige för ett projekt kan få det jobbigt om hen utöver sina vanliga arbetsuppgifter också ska vara ett stöd för mer juniora utvecklare (vilket är vanligt), ha ansvar för leveransen och dessutom sätta sig in i TDD samtidigt.

Mer än hälften av respondenterna påpekar att det är nödvändigt med självstudier. På frågan om man skulle kunna schemalägga dessa timmar på arbetstid säger respondent 4 att det till en viss del går men att det är “svårt att [schemalägga] så mycket tid som det egentligen skulle behövas”. Respondent 2 uttrycker samma tankar och påpekar vikten av att ha stöd från ledningen “för det kommer ta längre tid att utveckla systemet”. Hen fortsätter sedan med att “vissa kanske aldrig kommer att anamma TDD, men de kanske kommer att skriva enhetstester” och att det kanske är tillräckligt.

För att kunna införa TDD måste metoden ha stöd från berörda chefer. Chefer/ledningen måste ha en förståelse att metod kräver tid, pengar och resurser för att utvecklare ska få rätt utbildning och att seniora utvecklare ändå ska vara till hands när det behövs. Utan tillräckligt stöd från chefer/ledning kan det leda till att metoden gör mer skada än nytta. Detta medför också att utvecklarna måste ha rätt verktyg och rätt expertis för att snabbt kunna begripa metodiken.

4.3 Metodens roll

Här tar vi upp vad för roll metoden kommer att ha för utvecklaren under utvecklingsprocessen och vad för roller metoden har haft under tidigare projekt.

4.3.1 Standardisering av utvecklingsprocess

Fitzgerald (2002) menar att en standardisering av utvecklingsprocessen ökar utvecklarens förmåga för samordning och kommunikation inom komplexa projekt och att den samtidigt

underlättar underhåll och strukturering. Respondenterna är i olika grad medvetna om vad TDD tillför för utvecklingsprocessen och om problematiken med att följa principerna som finns i metodiken.

TDD är “ganska väl definierat och standardiserat som utvecklingsprocess” menar en respondent, och att principen med att skriva så lite kod som möjligt och sedan refaktorisera koden tills den är tillräckligt bra, är en bra grund. Respondenten anser också att hela värdet med TDD är att man driver fram en design av koden. På så sätt skapas klasser och funktioner som verkligen behövs. Som en av respondenterna påpekade, att kodningen ska gå “utifrån och in” istället för “från implementation och utåt”. Samtidigt som TDD är en väl definierad metod som den är, måste en utvecklare veta hur metoden ska användas för att ha en bra grund för att förstå hur metoden ska följas och fungera.

Respondenterna påpekade att svagheten hos TDD ligger i att testa större drag av ett system. Behöver en utvecklare testa flera lager av systemet är det svårt att testa större enheter som har externa beroenden. Vid sådana tillfällen anser respondenten att det är mer “okej” med automatiska enhetstester. Som en respondent formulerade sin definition: “TDD är lite som man gör det till”. Att man inte ska följa metoden så “himla religiöst”, som hen fortsatte, med att testa små enheter. Där ser inte respondenten någon fördel.

Flera av respondenterna påpekade också att problematiken med TDD är att metoden blir svåränvänd i vidareutveckling. De antydde att det blir för komplext och omständigt att försöka införa tester i något som redan finns. Det kan innebära att TDD inte är anpassat för att använda på något som redan finns, då man där till måste bryta ut all kod och göra om det. Respondenterna menade då att det krävs omskrivning av ett helt befintligt system där det saknas tester. Detta att bygga om gamla system kan bli en kostsam och tidskrävande process.

4.3.2 Systematisering av utvecklingskunskap

Fitzgerald (2002) menar att *utvecklingskunskap* är målet att ändra alla beroende av kunskaper till objektsform. Genom formaliserad kunskap inom metoden tillåter det att lagra, systematisera, sprida och utbyta kunskap. TDD överför kunskapen och låter den växa genom den testäckning den erbjuder.

TDD kan underlätta för en utvecklare, minska arbetsbördan med dokumentation och samtidigt se till att utvecklingen har gått till på rätt sätt. Respondent 4 menar att “... det dokumenteras för mycket. Med bra metod- och variabelnamn och med tester, så är det inte svårt att följa [koden]. Kopplingen mellan kod och dokumentation kommer aldrig matcha helt ändå på grund av ändringar som görs”.

Samma respondent förklarar vidare att personerna som hade svårt att se nyttan med TDD var “...så pass duktiga på sina projekt att de redan visste om hur systemet skulle reagera om de gör en ändring. Då behövs det inga tester. Men om personen skulle sluta är det jättesvårt för någon annan att sätta sig in i det. De är nästan lite själviska ... Ett argument är alltså att man ska [använda TDD] för andras skull”. “Om det bara är en person som vet vad han gjort är det en jätterisk, för det är ingen annan som förstår koden”, menade en annan respondent. Tester är ett sätt att dokumentera ens kunskap om systemet.

4.3.3 Möjliggöra projektstyrning & kontroll

Fitzgerald (2002) tar upp att metoder ska ge möjlighet till projektstyrning och kontroll över hur utvecklingsprocessen förbättras, möjligheten att granska framsteg, övervaka faktiska kostnader och jämföra och fördela med förväntade kostnader.

Det framhävdes att det är viktigt att chefer har en bra uppfattning vad TDD är och vilka fördelar det kan föra med sig. Flera av respondenterna tar upp att chefer måste vara medvetna om att TDD kommer kräva extra tid, pengar och resurser till en början och att utvecklare kommer att leverera mindre nytta under utvecklingsperioden. Något flera respondenter sedan menar att man tjänar in under förvaltningen.

4.3.4 Professionalisera SU-arbete

Det testdrivna arbetssättet förespråkar att utvecklaren tillämpar en viss utvecklingsstil. Till exempel måste utvecklaren skriva ett test först, och att det testet måste bli rött och så vidare (referera till Red/Green/Refactor under teoriavsnittet). Dessa är tydligt beskrivna arbetsregler och som "tvingar" utvecklaren att arbeta på ett kontrollerat sätt inom en cykel som repeteras för varje test under hela utvecklingen av systemet.

Respondent 2 påpekar en fördel med TDD när det gäller att testa generellt: "Om man skriver testerna i efterhand så kommer det en annan grej som är jättebråttom och då måste man hoppa över det och gör testerna nästa vecka istället och så blir det inget med det. Om man skriver testerna först kan ingen projektledare komma och säga 'nej, nu måste vi börja med det här istället'". Detta minskar risken för förvirring bland utvecklarna och där till får en bättre struktur på vad som måste prioriteras.

4.4 Utvecklingskontexten

Här tar vi upp utomstående faktorer som kan påverka en utvecklares förmåga att använda metoden och vad för komplikationer som metoden stöter på under utvecklingsprocessen.

4.4.1 Kontext och kultur

Stress, som uppstår på grund av hårda deadlines och krav på att leverera, har visat sig vara vanligt, speciellt hos de utvecklare som länge arbetat inom sin bransch. Ju mer leveranspress man har, desto vanligare är det att testerna nedprioriteras. "... stress gör att testerna får lida", menar en respondent.

Vanligtvis arbetar man med flera projekt samtidigt. En respondent berättar att denne hade tre olika projekt igång och förklarar vidare att det inte finns tillräckligt med tid för att kunna tillämpa TDD. "Man har för mycket att göra i huvudet, ibland dyker det upp andra saker man måste lära sig som kanske är viktigare än TDD", förklarar respondenten.

Det finns också en viss problematik med att arbeta testdrivet när kunden kräver tidig leverans av produkten. Vanligtvis vill kunden ha någon form av prototyp tidigt, men utvecklaren hinner sällan bli klar med den eftersom mycket av tiden läggs på att skriva tester, menar en respondent.

4.4.2 Filosofiska, praktiska och metodiska överväganden

Vid frågan om projektstorlek och projekttyp kan påverka om man väljer att tillämpa TDD i ett projekt eller inte fick vi svar om att det mestadels beror på utvecklarna i teamet. Det handlar alltså egentligen inte om vad det är för projekt man ska arbeta med, utan det gäller bara att hitta erfarna utvecklare (som länge arbetat med TDD) så att dessa personer kan föra vidare sin kunskap och hjälpa andra när de kör fast, menar en respondent.

“TDD går att applicera överallt, men det lämpar mer i större applikationer då dessa i regel är mer komplexa i sig och kostar mer pengar att bygga”, tillägger en respondent.

“TDD löser inte några problem, speciellt om det existerar andra problem i applikationen”, fick vi höra av en respondent. Det finns andra saker man måste ta itu med innan man kan börja fundera på TDD, tillade respondenten. Det är först och främst viktigare att man har kompetenta utvecklare, en kodbas som är enkel att förvalta och att man har koll på miljöer och rutiner i organisationen. Kundens krav är det viktigaste och detta är någonting som samtliga respondenter är enade om. “Men TDD är viktigare än de flesta tror, dock kan det vara svårt att mäta detta då TDD nedprioriteras ofta”, påpekade en annan.

4.5 Formaliserad metod

I detta avsnitt tar vi upp vad utvecklarna kan om metoden och vilka verktyg utvecklarna använder för att kunna använda TDD på ett korrekt sätt.

“Det är viktigt att hålla isär begreppen” säger en av respondenterna. En del menar nämligen att de arbetar testdrivet trots att man skriver kod före ett test. Så beroende på vem man frågar kan svaren skilja. En respondent säger till exempel “... bara för att någon säger att de skriver tester i NUnit så kör de TDD, men det stämmer inte alltid”.

Jeffries & Melnik (2007) förklarar TDD som en design- och programmeringsdisciplin där utvecklaren skriver ett test före koden. Testet förväntas i början bli rött, därefter implementerar utvecklaren tillräckligt med kod för att framkalla ett grönt test. Slutligen refaktorerar utvecklaren, där det finns möjlighet att finjustera koden.

En del utvecklare arbetar dock inte enligt den föreskrivna metoden, utan väljer att skräddarsy TDD för den givna situationen. En respondent föredrar till exempel att skriva stora (integrations) tester än små (enhets) tester, men testet skrivs fortfarande först. En del utvecklare skriver både integrationstester och enhetstester. En respondent hade gjort detta i ett tidigare projekt och tyckte att det var besvärligt eftersom man var tvungen att “mocka hit och mocka dit. TDD var inte anpassat för detta.”

Hur och när man ska tillämpa TDD i ett projekt råder det delade åsikter om. “Det gäller att göra det med sunt förnuft, någonting som kommer med erfarenhet”, menar en respondent. Det beror även på vilken roll utvecklaren har inom företaget. En respondent säger att man måste se till investeringskostnaden för TDD i förhållande till vilken typ av system som ska utvecklas och att “... i mindre projekt som måste utvecklas snabbt så kanske det inte är värt att arbeta testdrivet”. En annan respondent är inne på samma spår och menar att det är viktigt att hitta en bra balans och sedan göra en avvägning hur omfattande tester man ska ha, “... att inte göra

några test alls – det vet vi att det inte är lönsamt, det ger inte bra kundvärde. Men att skriva mycket tester är inte heller ett alternativ. Någonstans däremellan ligger *sanningen* ”.

En respondent, som är ny i yrket, tycker att TDD är bra delvis för att det blir lättare att se när man förstör logiken i programkoden (något test blir rött). Att börja med ett test som kommer att bli rött anses vara en kvalitetsfråga. “Det ger dig en indikation på att testet existerar och att du är på rätt väg”, påpekade en person.

4.5.1 Verktyg, tekniker och designmönster

En teknik nämns i samband med TDD under samtliga intervjuer: *mocking*. Det talades även mycket om designmönstret *dependency injection*. I korta drag används DI för att ta bort konkreta beroenden hos en klass. Klassen behöver därmed inte ansvara för att instantiera dess beroenden. Mock beskrivs som ett sätt att simulera eller “fejka” ett beteende hos en klass eller ett objekt.

DI var någonting som samtliga respondenter använde eller hade använt i något projekt, i samband med TDD. Det var till stor hjälp vid enhetstestning, tyckte alla. Många föredrog även att använda ett ramverk som stöd för DI, en så kallad *IoC-container*. Ett sådant ramverk håller bland annat koll på vilka beroenden en klass har och instantierar dessa automatiskt när klassen kräver det. Det vanligaste ramverket som användes bland respondenterna var Microsofts *Unity*.

Det finns även ramverk för *mocking*, men i och med att det är enkelt att skapa egna fejkade klasser så behöver man inget ramverk med extra funktioner, menade en respondent. “Det är trevligare att skapa egna mocks”, tillägger respondenten.

“Vissa gillar *mocking*, då driver man ut koden så att många objekt kommunicerar med varandra. Andra gillar det inte, då strukturerar man applikationen så att inga objekt kommunicerar med varandra (med hjälp av *dependency injection*) och endast får in det dom behöver utifrån”, sade en annan.

En respondent förklarar en fördel med att använda fakes och menar att testerna kan exekveras snabbare. Det finns en märkbar skillnad i att utföra test på verkliga objekt (ej fejkade) som kommunicerar med en databas (eftersom databasen först måste läsas av) i jämförelse med att köra tester på fejkade objekt. Man talar om en skillnad på några sekunder, vilket är mycket i detta sammanhang. I bilaga 3 har vi sammanställt ett exempel som visar denna skillnad.

4.6 Informationssystem

Ett informationssystem riktar sig till konstruktion, produktion och genomförande av ett system i en kontext men samtidigt har ett socialt och ett tekniskt synsätt. Analysen tar upp hur utvecklarerna tillämpar TDD för att konstruera ett informationssystem och vilka för- och nackdelar metoden har under utvecklingsprocessen.

TDD lämpar sig för små som stora projekt, enligt några respondenter. Men det är ännu bättre att applicera det på större projekt eftersom dessa i regel är mer komplexa, antydde en respondent. En annan menar att man kan avstå från TDD i små projekt och att det lämpar sig desto mer i projekt som ska förvaltas under en period på minst ett par år.

Respondenterna utesluter TDD inom vissa delar av ett system. Framförallt i applikationer med GUI-lager (Graphical User Interface, *grafiskt användargränssnitt*) och där GUI-lagret är starkt förenat med logiklagret. En respondent menar att det till och med är onödigt att applicera tester på ett GUI-lager: “Det är för besvärligt att göra det, så i mitt team struntar vi i det”.

Det är även svårt att applicera enhetstester på affärsregler som finns i klienten (Javascript till exempel) – där behövs dependency injection. Det finns lösningar för detta men man behöver göra en avvägning om man, förutom på serversidan, även behöver tester på klientsidan.

I regel lämpar det sig bättre att applicera enhetstester på operationer som returnerar ett konkret objekt och som inte är beroende av ett externt system. Ett praktexempel är beräkningsfunktioner som ofta enbart hanterar inmatning och utmatning av siffror.

För att kunna applicera TDD på ett befintligt system måste vissa delar bytas ut för att det ens ska vara möjligt att följa metoden.

Systemet förändras allteftersom tiden går, då användare kommer på nya sätt att använda systemet. Om kontexten förändras kommer designen för systemet förändras. En respondent tar upp att det blir “lättare att göra ändringar i systemet [med TDD], och det kommer även kunna leva längre för man kommer kunna refaktorera mycket lättare på grund av att man har sina tester”.

5 AVSLUTNING

Här kopplar vi det som kommit fram i resultat- och analysdelarna med vårt syfte och vår problemformulering och landar i ett antal slutsatser.

5.1 Slutsatser

Det huvudsakliga syftet med denna undersökning var att ta reda på hur en organisation kan lyckas med testdriven utveckling. Vi gjorde detta genom att från utvecklarens perspektiv identifiera faktorer som kan ligga bakom att organisationer och enskilda utvecklare har upplevt problem med TDD. Vi tittade sedan på hur och varför dessa faktorer påverkar TDD i användning.

Vi visar här hur method-in-action-molnet formas av de olika komponenterna i ramverket och våra slutsatser, formulerade som lösningsförslag, presenteras kategoriserat efter hur komponenterna i method-in-action-ramverket i förhållande till method-in-action-molnet påverkar varandra och framförallt TDD i användning.

Genom att följa dessa lösningsförslag har en organisation större möjlighet att lyckas med testdriven utveckling. Därför menar vi också att vårt syfte är uppfyllt.

5.1.1 Formaliserad metod kan utgöra grunden för en metod i användning

Här besvaras delfrågorna om hur utvecklarens utbildning, personliga kunskap om TDD samt programmeringsfärdighet påverkar TDD i användning.

Då det kan missförstås vad metoden TDD innebär är det svårt för utvecklare att skapa sig en klar bild av vad TDD är och inte är. Som att vissa utvecklare ser TDD-verktygen som TDD, vilket orsakar förvirring. För att omformulera det så följer inte utvecklarna teorin och istället så skraddarsys metoden. Metoden skapas då av de delar som passar förutsättningarna som utvecklaren ska ta ställning till under användning.

Dependency injection och mocking är någonting som ofta nämns i samband med TDD, och är en del av en utvecklarens vardag vid testdriven utveckling och därmed något som en utvecklare behöver sätta sig in i för att kunna arbeta med TDD. Vanligtvis använder utvecklaren något ramverk som stöd för detta. Tekniker och verktygsstöd spelar en stor roll hos TDD och är ett ämne man kan forska vidare på.

5.1.2 Metodens roll påverkar en metod i användning

Här besvaras delfrågorna om hur utvecklaren arbetar med TDD samt hur verktygsstöd påverkar TDD i användning.

TDD är i grund och botten ett standardiserat arbetssätt, som bland annat hjälper till att driva utvecklingen av systemets design. Detta rättfärdigar varför metoden ska användas. Det finns dock brister med metoden, exempelvis är det ytterst besvärligt att skriva enhetstester eller tillämpa TDD i redan existerande system. Tester kan även ses som ett sätt att dokumentera ett systems specifikationer.

5.1.3 Metodens roll rättfärdigar formaliserad metod

Här besvaras delfrågan om hur stöd från chef och ledning påverkar TDD i användning.

Kunder och ledning måste vara medvetna om att det krävs planering, tid och ekonomiska resurser för att lyckas med TDD. De måste därför se nyttan av att ha en högre utvecklingskostnad, men i gengäld få en lägre förvaltningskostnad. Man tjänar på att tänka proaktivt. Speciellt när de annars kanske hade varit tvungna att köpa in ett helt nytt system då det gamla inte går att anpassa efter förändrade förhållanden såsom nya krav på funktionalitet, stabilitet eller säkerhet. I vissa utvecklingsteam värderar man andra saker högre än att ha en bra testtäckning. Att framförallt ha kompetenta utvecklare i ett team, kontroll på arbetsmiljöer och rutiner och en bra grund för att utveckla applikationen i, är bara några exempel.

5.1.4 Utvecklingskontexten formar en metod i användning

Här besvaras delfrågorna om hur stöd från chef och ledning, utvecklarens programmeringserfarenhet samt stress eller leverenskrav påverkar TDD i användning.

Om organisationen huvudsakligen är konsulter åt andra gäller det att visa för kunderna att de kommer ha nytta av att prioritera TDD. Om organisationen huvudsakligen ägnar sig åt utveckling av egna system så är det istället ledningen man behöver få med sig. Om man tagit fasta på detta så kommer TDD att användas i fler projekt. Det leder till att fler utvecklare får allt större praktisk erfarenhet av TDD, något som vi sett saknas till stor del bland samtliga respondenter.

Om en person har andra arbetsuppgifter utöver utvecklingsarbetet är det viktigt att titta på den personens ansvar och organisationens förväntningar på densamme. Om denna person ska lära sig TDD får det oftast göras på bekostnad av någonting annat.

Utvecklare som arbetat länge inom sin bransch upplever oftast mer stress än de nyutbildade. Detta kan bero på att man (som erfaren utvecklare) oftast arbetar med flera projekt samtidigt och är ytterst ansvarig för leveransen av ett projekt. Detta medför vanligtvis att testerna inte prioriteras som första hand, menar en respondent.

5.1.5 Utvecklare analyserar utvecklingskontexten

Här besvaras delfrågorna om hur stöd från chef och ledning, utvecklarens personliga inställning samt stress eller leverenskrav påverkar TDD i användning.

Hur man inför TDD i en organisation är av yttersta vikt. Kommer det som ett krav uppifrån (chefer och/eller ledning), utan att vara förankrat i verksamheten och utan att utvecklarna är insatta i det, kommer engagemanget för TDD att störtdyka.

Att förvänta sig att en utvecklare bedriver självstudier på fritiden blir ett problem då det kan finnas faktorer i en persons liv som gör att det inte är möjligt eller troligt att det blir av – vare sig det beror på barn, vård av annan person, andra engagemang kvällstid eller en hobby.

De som planerar ett inlärningsprogram i en organisation bör ha i åtanke att det krävs olika upplägg beroende på utvecklarens bakgrund. Det finns ingen universallösning.

Nyexaminerade lär sig på ett sätt, seniora utvecklare som är sugna på att ta till sig ny teknik lär sig på ett annat och seniora som har mindre förändringsvilja lär sig på ytterligare ett sätt.

Ett möjligt inlärningsprogram är att:

- 1) Läsa teori. Kurs eller självstudier. Även designmönster såsom DI och mockobjekt.
- 2) Se till att en mentor finns tillgänglig på plats.
- 3) Projekt att tillämpa teorin på. Bör vara ett enklare projekt, med mest get-/set-metoder. Det vill säga enkel inmatning, utmatning av data.
- 4) Låt det ta tid. För det handlar i grund och botten om att ändra inställning och det kan man inte göra genom en 3-dagars kurs.

5.1.6 Utvecklare utför en metod i användning

Här besvaras delfrågorna om hur utvecklarens personliga inställning samt programmeringserfarenhet påverkar TDD i användning.

Utvecklare med ett par års erfarenhet med TDD har benägenhet att “skraddarsy” metoden och applicerar olika delar av TDD i sina egna projekt snarare än att tillämpa metoden fullt ut (enligt teorin). Samtliga utvecklare vi intervjuade hade börjat med någon form av teoribildning innan de började att arbeta praktiskt med TDD. För att skapa någon nytta med TDD behöver man inte bara förstå det, man måste även använda det på ett bra sätt - det vill säga kunna skriva bra tester och veta i vilka sammanhang som den formella metoden bör begränsas. För att förstå teorin måste utvecklaren tillbringa mer tid för att få en bättre, mer konkret förståelse för metoden.

5.1.7 Utvecklare framställer informationssystem

Här besvaras delfrågan om hur typ av applikation som ska utvecklas påverkar TDD i användning.

Vinsten av att tillämpa det testdrivna arbetssättet är mer märkbart vid utveckling av större applikationer (exempelvis enterprise applikationer), då dessa i regel är mer komplexa samt kostar mer pengar att konstruera, påpekar en respondent, som är konsult inom sitt företag. “Tester är som skyddsplankor”, tillägger en annan respondent.

TDD, men framförallt enhetstester, är svårt att applicera på affärslogik som ligger på klienten, till exempel JavaScript. Det lämpar desto bättre att genomföra enhetstester på metoder eller funktioner som har ett tydligt syfte (läs om *Single Responsible Principle*). Metoder med stark isolering (fristående från yttre beroenden) och som returnerar en värdetyp är ett praktexempel på detta.

5.2 Egna reflektioner

Några reflektioner som vi vill påpeka är att våra kunskaper med Method-In-Action i början av uppsatsen var relativt grundläggande. Intervjufrågorna som var baserade på Method-In-Action skulle formuleras på ett annat sätt ifall vi hade haft mer kunskap med ramverket när intervjuerna genomfördes. Detta skulle kanske gett mer konkret data för vissa komponenter i ramverket.

En annan aspekt som framkom var att vi skulle haft en inledande intervju på området med en TDD-expert. Detta skulle ge en bättre förståelse för hur metoden fungerar praktiskt och det skulle möjligtvis gett oss ett mer tekniskt synsätt på processen.

5.3 Vidareforskning

En vidarestudie som ämnar att undersöka huruvida man kan utbilda utvecklare inom TDD på bästa sätt är ett forskningsområde vi tycker verkar intressant, men som vi också anser kan vara ett hjälpmedel för de utvecklare som precis ska börja lära sig TDD.

Hur man kan refaktorera existerande system och samtidigt tillämpa det testdrivna arbetssättet beskrivs i allt för lite omfattning i de litteraturer vi läst och är någonting som vi, men även större organisationer som dagligen arbetar med att underhålla och vidareutveckla system kan dra nytta av.

KÄLLFÖRTECKNING

- ❖ Astels, D. (2003). *Test-driven development*. Upper Saddle River: Prentice Hall.
- ❖ Beck, K. (2003). *Test-driven Development: By Example*. Boston: Pearson Education.
- ❖ Bryman, A. (2002). *Samhällsvetenskapliga metoder*. Malmö: Liber.
- ❖ Buffardi, K. & Edwards, S. H. (2012). Exploring influences on student adherence to test-driven development. In *Proceedings of the 17th ACM annual conference on Innovation and technology in computer science education (ITiCSE '12)*, New York, 2012-07-03, 105-110. doi:10.1145/2325296.2325324
- ❖ Buschmann, F. (2011). Tests: The architect's best friend. *IEEE Software*, 28(3), 7-9.
- ❖ Crispin, L. (2006). Driving Software Quality: How Test-Driven Development Impacts Software Quality. *IEEE Software* 23(6), 70-71. doi:10.1109/MS.2006.157
- ❖ Ebert, C. (2003). Software Quality Management. I M. A. Drake (Red.), *Encyclopedia of Library and Information Science, Second Edition* (s. 2674-2687). New York: Marcel Dekker, Inc.
- ❖ Fitzgerald, B., Russo, N. L. & Stolterman, E. (2002). *Information Systems Development: Methods-in-Action*. Berkshire: McGraw-Hill Education.
- ❖ Fowler, M. (1999). *Refactoring: Improving The Design Of Existing Code*. Boston: Addison-Wesley Longman Inc.
- ❖ Galloway, J., Haack, P., Wilson, B. & Allen, K. S. (2012). *Professional ASP.NET MVC 4*. Indianapolis: John Wiley & Sons, Inc.
- ❖ Goldkuhl, G. (1991). Stöd och struktur i systemutvecklingsprocessen. På konferensen *Systemutveckling i praktisk belysning*, Norrköping, 1991-11-19.
- ❖ Goldkuhl, G. (1993). Välgrundad metodutveckling. På *VITS Höstseminarium*, Linköping, 1993-09-28/29.
- ❖ Goldkuhl, G. (2011). *Kunskapande*. Linköping: Linköpings universitet.
- ❖ Hammond, S. & Umphress, D. (2012). *Test driven development: the state of the practice*. In *Proceedings of the 50th Annual Southeast Regional Conference (ACM-SE '12)*, New York, 2012-03-29, 158-163. doi:10.1145/2184512.2184550
- ❖ Janzen, D. & Saiedian, H. (2005). Test-driven development concepts, taxonomy, and future direction. *Computer*, 38(9), 43-50. doi: 10.1109/MC.2005.314
- ❖ Jeffries, R. & Melnik, G. (2007). TDD: The Art of Fearless Programming. *IEEE Software*, 24(3), 24-30. doi:10.1109/MS.2007.75
- ❖ Kollanus, S. (2010). *Test-Driven Development – Still a Promising Approach?* Paper presented at the 2010 Seventh International Conference on the Quality of Information and Communications Technology (QUATIC), Porto, 2010-09-29–2010-10-02, 403-408. doi: 10.1109/QUATIC.2010.73

- ❖ Marchesi, M. & Succi, G. (2003). *Extreme Programming and Agile Processes in Software Engineering*. Berlin: Springer.
- ❖ Martin, R. C. (2007). Professionalism and test-driven development. *IEEE Software*, 24(3), 32. doi:10.1109/MS.2007.85
- ❖ Martin, R. C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Upper Saddle River: Prentice Hall.
- ❖ Lui, K.M. & Chan, K.C.C. (2008). *Software Development Rhythms: Harmonizing Agile Practices for Synergy*. Hoboken: John Wiley & Sons, Inc.
- ❖ McDaid, K., & Rust, A. (2009). Test-driven development for spreadsheet risk management. *IEEE Software*, 26(5), 31-36. doi:10.1109/MS.2009.143
- ❖ Osherove, R. (2013). *The Art of Unit Testing*. Greenwich: Manning Publications Co.
- ❖ Shihab, E., Jiang, Z. M., Adams, B., Hassan, A. E. and Bowerman, R. (2011), Prioritizing the creation of unit tests in legacy software systems. *Software: Practice and Experience*, 41(10), 1027–1048. doi: 10.1002/spe.1053
- ❖ Shore, J. & Warden, S. (2008). *The art of agile development*. Sebastopol: O'Reilly.
- ❖ Thurén, T. & Krisberedskapsmyndigheten. (2003). *Sant eller falskt? Metoder i källkritik*. Stockholm: Krisberedskapsmyndigheten.
- ❖ Tort, A., Olivé, A. & Sancho, M-R. (2011). An approach to test-driven development of conceptual schemas. *Data & Knowledge Engineering*, 70(12), 1088-1111. doi:10.1016/j.datak.2011.07.006
- ❖ Vetenskapsrådet. (2002). *Forskningsetiska principer inom humanistisk-samhällsvetenskaplig forskning*. Uppsala: Vetenskapsrådet.

BILAGOR

Bilaga 1, Intervjufrågor

Utgångsläge och perspektiv

1. Ditt namn?
2. Hur många år har du totalt jobbat som systemutvecklare?
3. Vad har du för arbetstitel i dagsläget? (Junior, senior, lead developer...)
4. Vad är din definition av TDD? Vilket ord beskriver bäst TDD?
5. Hur skulle du själv säga att din inställning till TDD ser ut?

Inläarning

6. Vilken utbildning inom TDD har du tidigare fått? Hur lärde du dig TDD?
7. Hur lång tid tog det från den dagen du började lära dig TDD tills du kände att du var tillfredsställd med dina kunskaper i TDD?
8. Vad skulle du ändra i inlärningsprocessen om du skulle ge rekommendationer till andra som ska lära sig TDD?

Användning

9. I hur många projekt har du arbetat med TDD i någon utsträckning? Det vill säga skrivit ett test som blivit rött, sen skrivit kod för att få testet grönt, sedan refaktorering och så vidare.
10. I hur många projekt har du arbetat med automatiserade tester? Det vill säga ej "ren" TDD men med en del av TDD. Att man skriver produktionskod först och sedan testar.
11. Vilken systemutvecklingsmetodik tillämpades när du använde TDD?
12. Fanns det något med TDD som du tyckte var svårt eller omständigt?
13. Hur kontrollerar ni kod eller program innan driftsättning? Vad för typer av kod-tester använder ni i dagsläget?
14. Hur ser du på skillnaden att arbeta enligt TDD och att arbeta på traditionellt vis?
15. Vilka svårigheter eller nackdelar upplever du med att arbeta med TDD jämfört med ditt vanliga arbetssätt? Hur tror du att det kommer sig?
16. Vilka fördelar upplever du med att arbeta med TDD jämfört med ditt vanliga arbetssätt? Hur tror du att det kommer sig?
17. Hur skulle du vilja förändra TDD om du fick ta bort eller lägga till komponenter?
18. Tycker du att det finns några projekt där TDD passar bra att använda eller några där man bör undvika det? Små eller stora, komplexa system, lösa svåra matematiska algoritmer eller liknande. Varför?
19. Anser du att projektstorlek påverkar användandet av TDD?
20. Ni har testat TDD men inte infört det i all typ av utveckling. Varför?
 - a. Finns det några aspekter som du tror kan ha lett till detta?
 - b. Vad skulle du göra för att få det att fungera nästa gång?

21. Hur tror du man kan få utvecklare att acceptera TDD?
22. Hur anser du att man bör tillämpa TDD för att det ska accepteras i organisationen?
Tror du att det finns något "rätt" sätt att tillämpa TDD på eller ett "typiskt" arbetssätt enligt dig?
23. Är det viktigt att alla som arbetar med projektet är med på att använda TDD eller räcker det med att det finns enstaka personer som kan använda sig av det?

Bilaga 2, Kodexempel: Hur stub och mock kan användas för att skriva automatiserade tester (i ramverket NUnit).

```
public class User
{
    public int UserId { get; set; }
    public string Name { get; set; }
}
```

```
public interface IUserRepository
{
    IEnumerable<User> GetAll();
}
```

```
public class UserRepository : IUserRepository
{
    private UserContext _context;

    public IEnumerable<User> GetAll()
    {
        using (_context = new UserContext())
        {
            return _context.Users.ToList();
        }
    }
}
```

```

public class Consumer
{
    private IUserRepository _userRepository;

    public Consumer(IUserRepository userRepository)
    {
        _userRepository = userRepository;
    }

    public IList<User> GetUsers()
    {
        return _userRepository.GetAll().ToList();
    }
}

```

```

[TestFixture]
public class UserTests
{
    [Test]
    public void Should_Get_Three_Users_From_Database()
    {
        var consumer = new Consumer(new UserRepository());

        var result = consumer.GetUsers();

        Assert.AreEqual(result.Count, 3);

        Testet utförs direkt på klassen Consumer och dess konkreta beroende: UserRepository
    }

    [Test]
    public void Should_Get_Three_Users_Using_Stub()

```

```

{
    var consumer = new Consumer(new UserRepositoryStub());

    var result = consumer.GetUsers();

    Assert.AreEqual(result.Count, 3);

```

Testet utförs även här på klassen Consumer men vi testar inte längre dess konkreta beroende utan ersätter den med en stub: UserRepositoryStub.

```

}

[Test]
public void Should_Get_Three_Users_Using_Mock()
{
    var mock = new UserRepositoryMock(new UserRepositoryStub());

    var result = mock.GetUsers();

    Assert.AreEqual(result.Count, 3);

    Vi testar inte längre klassen Consumer utan istället på ett mockobjekt som i sin tur utnyttjar en stub.

}
}

```

```

public class UserRepositoryStub : IUserRepository
{
    public IEnumerable<User> GetAll()
    {
        var users = new List<User>()
        {
            new User() { },
            new User() { },
            new User() { }
        };
    }
}

```

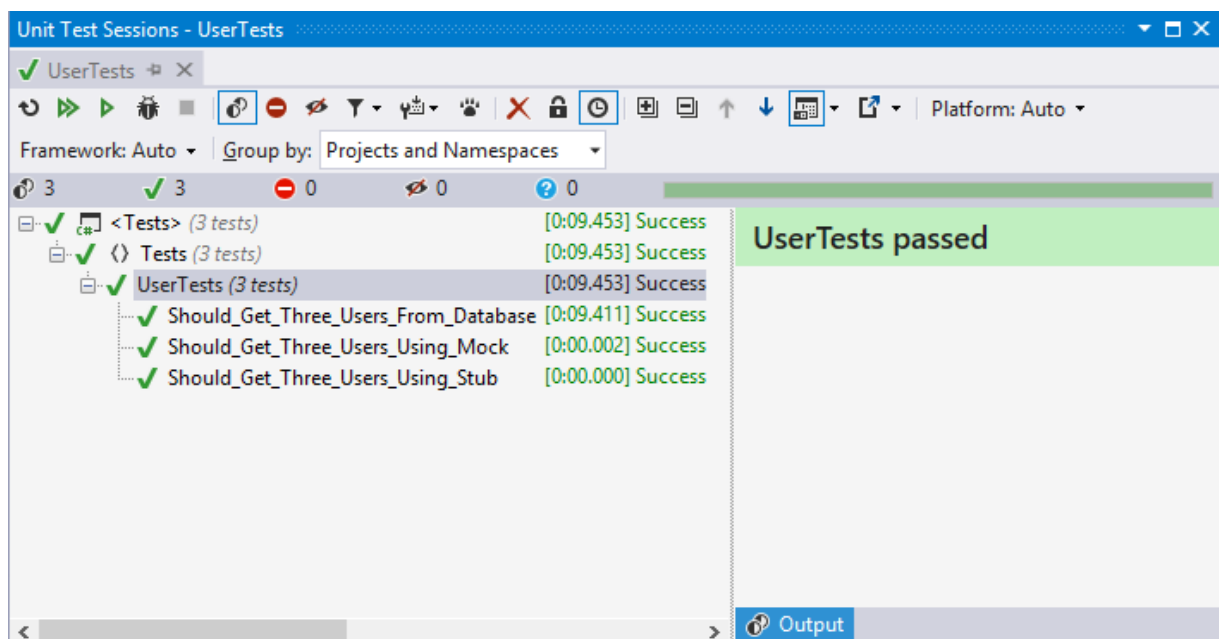
```
        return users;
    }
}
```

```
public class UserRepositoryMock
{
    private IUserRepository _userRepository;

    public UserRepositoryMock(IUserRepository userRepository)
    {
        _userRepository = userRepository;
    }

    public IList<User> GetUsers()
    {
        return _userRepository.GetAll().ToList();
    }
}
```

Resultat



Att köra tester mot objekt som kommunicerar med en databas (typiska integrationstester) tar lite mer

än 9 sekunder. Tester som körs mot fakes exekveras nästan omedelbart.

Bilaga 3, Kodexempel: Hur man applicerar dependency injection på kod

```
public class Authenticator
{
    public bool Login(string username, string password)
    {
        var validator = new Validator();

        return
            validator.ValidateUsername(username) &&
            validator.ValidatePassword(password);
    }
}
```

Klassen Authenticator är beroende av Validator.

```
public class Validator
{
    public bool ValidatePassword(string password)
    {
        return password != null;
    }

    public bool ValidateUsername(string username)
    {
        return username != null;
    }
}
```

```
}
```

I nedanstående exempel använder vi Dependency Injection för att eliminera det konkreta beroendet hos klassen Authenticator. Vi skapar därför ett abstraktlager: IValidator.

```
public interface IValidator
{
    bool ValidatePassword(string password);
    bool ValidateUsername(string username);
}
```

```
public class Validator : IValidator
{
    public bool ValidatePassword(string password)
    {
        return password != null;
    }

    public bool ValidateUsername(string username)
    {
        return username != null;
    }
}
```

```
public class Authenticator
{
    private IValidator _validator;
```

```

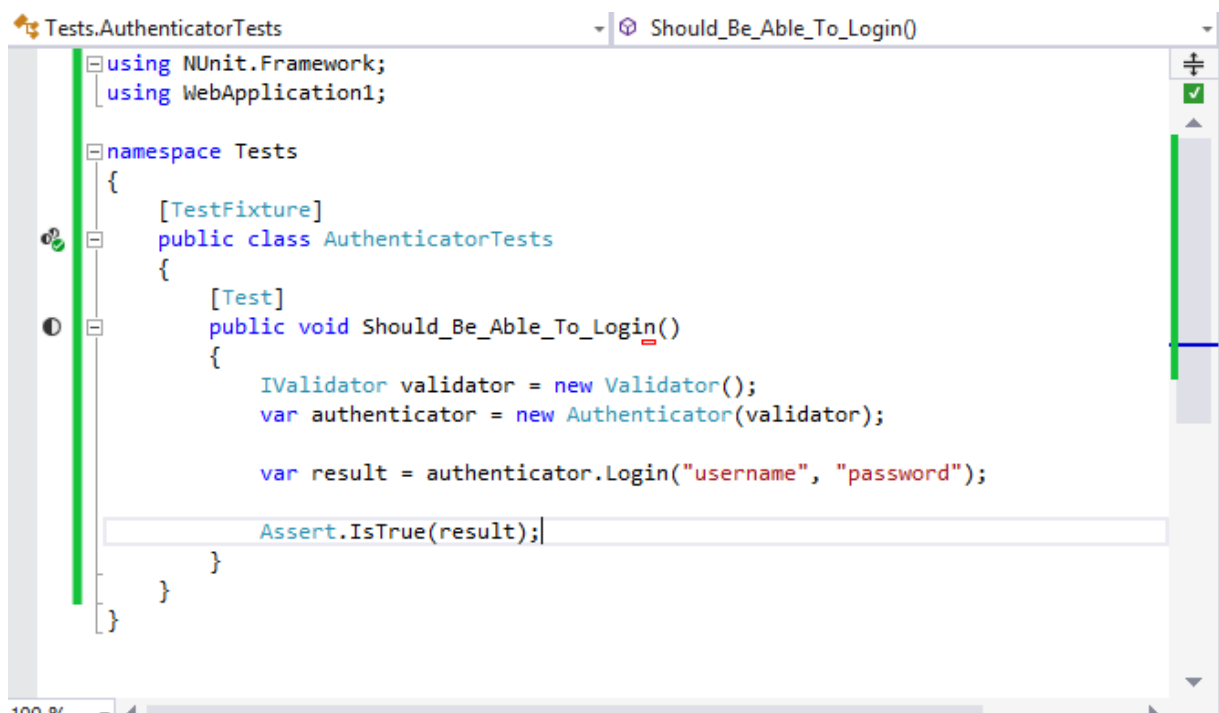
public Authenticator(IValidator validator)
{
    _validator = validator;
    Klassen ansvarar inte längre för att instatera Validator, den kräver istället en instans av det via dess konstruktor. Detta kallas för en Constructor Injection.
}

public bool Login(string username, string password)
{
    var validator = new Validator();

    return
        _validator.ValidateUsername(username) &&
        _validator.ValidatePassword(password);
}
}

```

Authenticator kan nu användas på följande sätt.



Notera att det är testklassen (AuthenticatorTests) som ansvarar för att instantiera Validator och inte längre klassen Authenticator.

I real-world applikationer använder man oftast ett ramverk (IoC container) för att konfigurera och

hantera instatieringen av olika beroenden hos en klass, instantieringen sker därmed automatiskt. I nedanstående exempel använder vi Unity.

En typisk konfiguration med Unity kan se ut såhär:

```
private static IUnityContainer BuildUnityContainer()
{
    var container = new UnityContainer();

    container.RegisterType<IValidator, Validator>();
    RegisterTypes(container);

    return container;
}
```